

Capítulo 1 - Introdução a programação Mobile

Bem vindos alunos.

Nesta apostila iremos tratar de métodos e formas de desenvolver aplicativos para dispositivos móveis.

Dispositivos móveis.

Dispositivos móveis são aparelhos que por definição são portáteis, possuem acesso à rede e um sistema operacional.

Hoje podemos dizer que qualquer smartphone ou tablet se encaixa nesta descrição.

Vamos começar esta introdução falando sobre os sistemas operacionais e suas linguagens de programação nativas.

Sistemas operacionais e suas linguagens.

Todos os dispositivos móveis disponíveis no mercado possuem o que chamamos de sistema operacional. Imaginamos que você, meu caro leitor, já esteja familiarizado com este conceito, porém vamos lembrá-los rapidamente do que se trata.

Um sistema operacional é um conjunto de programas criados para facilitar a interação entre o usuário e o hardware da máquina em questão, no nosso caso, os dispositivos móveis.

Vamos falar rapidamente sobre alguns dos principais sistemas operacionais dos dispositivos móveis presentes no mercado.

Android.

O sistema operacional Android foi fundado pela empresa Android.inc na Califórnia no ano de 2003.

No ano de 2005 a Google adquiriu esta empresa e a adicionou ao seu grupo empresarial na intenção de adentrar ao mercado de telefones moveis que no momento estava ainda em seu inicio.



Por volta de 2007 o Google e suas empresas parceiras no desenvolvimento de dispositivos móveis como a Samsung, HTC, Sony entre outras, lançaram o Android como uma plataforma aberta para uso em todos os seus futuros aparelhos.

O Android possui em seu princípio, várias contribuições do sistema operacional Linux, aliando isso ao fato de que o sistema foi concebido para ser aberto, o mesmo é considerado um software de código aberto.

Hoje o Android é o sistema operacional mais utilizado no mundo, estando presente em uma larga maioria dos smartphones e tablets do mundo.

A linguagem considerada padrão para o desenvolvimento no Android é o Java, uma linguagem muito conhecida entre os desenvolvedores por sua versatilidade. Tal condição ocorre em conjunto com o Android SDK.



Android SDK

iOS.

O iOS é o sistema operacional criado pela Apple, uma das grandes empresas de tecnologia do mundo.

O iOS é utilizado em smartphones e tablets produzidos exclusivamente pela Apple criando assim um ambiente fechado de desenvolvimento e aplicação.

O iOS foi anunciado por volta do ano de 2007, tendo seu lançamento oficial em no ano de 2008.



Entre suas características está que o mesmo tem uma integração muito superior ao hardware em que opera, já que o mesmo foi desenvolvido para aquele hardware diferentemente de seus concorrentes que objetivam funcionar em aparelhos produzidos por diversas empresas. Esta característica em especial é de grande valia tendo em vista que o

desempenho do aparelho em relação ao hardware é visivelmente superior.

Outra característica é que a Apple controla com muito mais cuidado os aplicativos que poderão ser executados em sua plataforma dando mais segurança, porém criando uma camada a mais de burocracia para os desenvolvedores lançarem seus aplicativos.

A linguagem de programação oficial para o desenvolvimento nas plataformas iOS é o Object-C Swift.

Aplicativos nativos, híbridos e webapps.

Agora falaremos sobre três métodos diferentes de programar aplicativos móveis: Nativo, híbrido e o webapp.

Nativo:

São aqueles aplicativos que são desenvolvidos na linguagem oficial do sistema operacional. Exemplo: Object-C Swift para o iOS ou o Java para o Android.

Vantagens:

- Velocidade de execução.
- Performance melhorada.
- Melhor acesso ao hardware do aparelho (câmera, GPS, etc...)

Desvantagens:

- Necessidade de conhecer linguagens de programação diferentes para cada plataforma.
- Reprogramar o mesmo aplicativo em cada linguagem necessária para as múltiplas plataformas.

Webapp:

As webapp's são aplicações feitas em linguagens WEB com o seu layout preparado para executar na tela do móbile. Na pratica é apenas um site que possui um atalho no celular de seu usuário.

Vantagens:

- Velocidade no desenvolvimento.
- Portabilidade para múltiplos dispositivos.
- Atualização dinâmica.

Desvantagens:

- Baixa performance.
- Praticamente nenhum acesso direto aos recursos de hardware (dependendo do navegador para tal).
- Não é permitida a postagem em nenhuma loja de vendas de aplicativos dos sistemas operacionais.

Híbrido:

Os aplicativos híbridos são uma mistura entre as aplicações nativas e webapp. Os mesmos são construídos utilizando partes de códigos web tradicionais, mas rodando sobre um aplicativo construído de forma nativa.

Normalmente este tipo de aplicativo possui uma parte básica construída de forma nativa e a parte que contém a lógica de negócio em forma de web.

Vantagens:

- Um pouco mais rápida de desenvolver que o nativo, tendo em vista que apenas uma parte é nativa.
- Velocidade de atualização, vendo que a parte que esta na web pode ser atualizada imediatamente sem necessidade de download do cliente.
- Mais acesso ao hardware que o webapp.

Desvantagens:

- Performance inferior ao nativo.
- Menos acesso ao hardware que o nativo.
- Velocidade inferior ao nativo.

Com isso finalizamos este inicio sobre a programação de aplicativos mobiles. Espero que tenham entendido e nos vemos no próximo capítulo.

Capítulo 1: Introdução à programação Mobile

NOME: _____ N° _____

TURMA: _____

PROFESSOR: _____

DATA: __/__/____

NOTA:

Exercícios

1 – Defina seu entendimento sobre os sistemas operacionais móveis.

2 – Um sistema operacional móvel possui uma linguagem nativa, quais as linguagens nativas dos maiores sistemas operacionais do mercado atual?

3 – No seu ponto de vista qual a melhor abordagem de desenvolvimento para aplicativos móveis com grandes quantidades de recursos disponíveis (Nativo, webapp, Híbrido)? Justifique.

4 – No seu ponto de vista qual a melhor abordagem de desenvolvimento para aplicativos móveis com poucos recursos disponíveis (Nativo, webapp, Híbrido)? Justifique.

5 – Em sua opinião existe alguma abordagem que não deva ser utilizada sob nenhuma circunstância no mercado atual? Justifique.

Atividade Prática: 01

1 – Crie grupos e apresente a classe e ao professor situações onde os métodos: híbrido, nativo, webapp, são ideias para o desenvolvimento.

Capítulo 2: Introdução ao React Native e Expo

1.1 O que é React Native?

React Native é uma biblioteca de código aberto criada pelo Facebook que permite desenvolver aplicativos móveis usando JavaScript e React. A principal vantagem do React Native é a capacidade de escrever código em JavaScript e renderizá-lo em componentes nativos, oferecendo uma experiência próxima à de aplicativos desenvolvidos nativamente para iOS e Android.



Vantagens do React Native

- **Reutilização de Código:** Um dos maiores benefícios do React Native é a capacidade de reutilizar código entre diferentes plataformas, economizando tempo e esforço.
- **Desempenho Nativo:** Ao contrário de outras soluções híbridas, o React Native usa componentes nativos, resultando em um desempenho mais próximo ao nativo.
- **Comunidade e Ecossistema:** Como uma biblioteca popular, React Native possui uma grande comunidade e um ecossistema robusto de bibliotecas e ferramentas.

1.2 Diferenças Entre Desenvolvimento Nativo e React Native

Antes de mergulharmos no React Native, é importante entender as principais diferenças entre desenvolvimento nativo e React Native.

Desenvolvimento Nativo

- **Linguagens:** Utiliza linguagens específicas para cada plataforma, como Swift ou Objective-C para iOS e Java ou Kotlin para Android.
- **Desempenho:** Geralmente oferece o melhor desempenho possível, pois o código é compilado diretamente para código de máquina.
- **Acesso a APIs Nativas:** Tem acesso completo a todas as APIs nativas e funcionalidades do dispositivo.

Desenvolvimento com React Native

- **Linguagem:** Usa JavaScript e React, permitindo que desenvolvedores web migrem para o desenvolvimento móvel mais facilmente.
- **Desempenho:** Oferece um bom desempenho ao usar componentes nativos, mas pode ter algumas limitações em comparação com código nativo puro.
- **Reutilização de Código:** Permite a reutilização de uma grande parte do código entre iOS e Android, reduzindo significativamente o tempo de desenvolvimento.

1.3 Introdução ao Expo

Expo é um conjunto de ferramentas e serviços construídos em torno do React Native que facilita o desenvolvimento de aplicativos móveis. Ele fornece uma série de bibliotecas pré-configuradas e uma CLI (Command Line Interface) que simplifica o processo de desenvolvimento.



Expo

Por que Usar Expo?

- **Facilidade de Configuração:** Com Expo, não há necessidade de configurar um ambiente de desenvolvimento nativo para iOS ou Android. Tudo o que você precisa é de um editor de texto e o Expo CLI.
- **Desenvolvimento Rápido:** Expo fornece uma série de ferramentas para desenvolvimento rápido, como a possibilidade de recarregar a aplicação instantaneamente após mudanças no código.
- **API Ricas:** Expo vem com uma série de APIs integradas para acessar funcionalidades do dispositivo, como câmera, localização, notificações, e muito mais.

1.4 Configuração do Ambiente de Desenvolvimento

Vamos agora configurar nosso ambiente de desenvolvimento para começar a trabalhar com React Native e Expo.

//passo a passo da instalação do java11 e do android studio.

Passo 1: Instalar Node.js

Antes de mais nada, precisamos instalar o Node.js, que inclui o npm (Node Package Manager). O npm é usado para gerenciar as dependências do projeto.

- Baixe e instale a versão mais recente do Node.js a partir do site: <https://nodejs.org/en>



Passo 2: Baixar bibliotecas

Como parte do processo, é inteligente baixar algumas bibliotecas, basta executar o comando abaixo no terminal de comando do windows:

```
npx expo install react-dom react-native-web @expo/metro-runtime
```

Este conjunto de comandos irá executar o download das bibliotecas necessárias para simulações locais.

Passo 3: Criar um Novo Projeto Expo

Agora, vamos criar um novo projeto Expo. Abra o terminal, navegue até a pasta que deseja que o projeto esteja e digite o comando abaixo:

```
npx create-expo-app meuprojeto --template blank
```

A parte sublinhada deve ser trocada pelo nome do projeto que deseja

Vale ressaltar que, o comando *--template blank*, indica que estamos criando um projeto sem conteúdo, existem outros modelos de template, porém como pretendemos estudar todo o processo de criação de um app, usaremos o *blank* mesmo.

Seria interessante que, a partir de agora, você deve entrar na pasta criada pelo comando anterior, onde o projeto está. Devemos lembrar

que, esta pasta possui todos os arquivos do projeto, logo também será o local onde abriremos e emitiremos os arquivos do código.

Para isso, vamos entrar na pasta com o velho comando:

```
cd meuprojeto
```

Passo 4: Executar a Aplicação com Expo Go

Expo Go é um aplicativo móvel disponível na App Store e Google Play que permite visualizar o seu aplicativo Expo em tempo real.

- Instale o Expo Go no seu dispositivo móvel.
- No terminal, inicie o servidor de desenvolvimento Expo:

```
npx expo start
```



Um QR code será exibido no terminal. Escaneie o QR code com o aplicativo Expo Go para visualizar e interagir com sua aplicação.

1.5 Primeira Aplicação "Hello World"

Vamos criar uma aplicação simples "Hello World" para nos familiarizarmos com a estrutura de um projeto React Native com Expo.

Entendendo o App.js

Dentro do arquivo **App.js** temos o seguinte código:

```
import { StatusBar } from 'expo-status-bar';
import { StyleSheet, Text, View } from 'react-native';

export default function App() {
  return (
    <View style={styles.container}>
      <Text>Open up App.js to start working on your app!</Text>
      <StatusBar style="auto" />
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    alignItems: 'center',
    justifyContent: 'center',
  },
});
```

Temos três blocos de código aqui, o primeiro composto de imports, no caso estamos importando o StatusBar, StyleSheet, Text, View.

Com essas importações importações, vamos falar de alguns componentes:

- **<View>** é o conjunto principal de uma página dentro do reactNative. No geral não é possível ter mais de uma por página. No exemplo acima, a mesma possui um parâmetro,

“style={styles.container}” esse parâmetro está fazendo o equivalente a dar um nome a este objeto para receber propriedades CSS no futuro.

- **<Text>** Neste componente, temos a escrita na tela, deste componente podemos colocar um conteúdo para ser impresso.
- **<StatusBar>** É um componente utilizado para controlar a aparência da barra de status do dispositivo. A barra de status é a área na parte superior da tela do dispositivo que exibe informações como a hora, a bateria, e o sinal da rede. A Configuração presente no exemplo é ‘style="auto"' porém no nosso código não temos nenhuma configuração para essa folha de estilo, logo ele irá manter as configurações padrão.
- **StyleSheet.create({ ... })**: Cria um objeto de estilos que pode ser usado para estilizar os componentes, como podemos ver no exemplo, é criado o ‘container’ que está associado ao “view”.

Conclusão

Neste capítulo, você foi introduzido ao React Native e ao Expo, configurou seu ambiente de desenvolvimento e criou sua primeira aplicação "Hello World". Nos próximos capítulos, mergulharemos mais fundo nos conceitos de React Native, explorando componentes, navegação, gerenciamento de estado e muito mais.

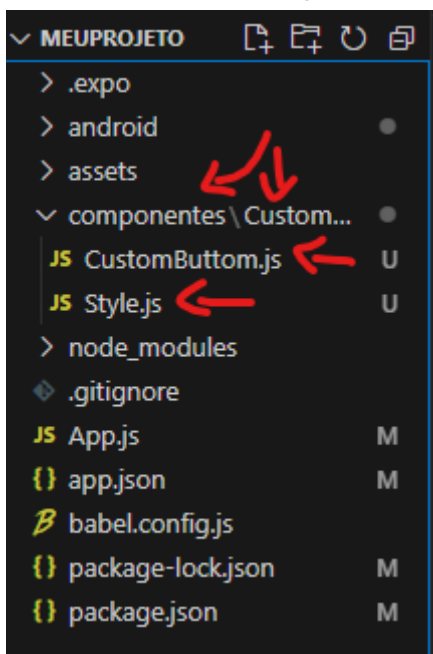
Capítulo 3: Componentes externos

É da natureza da programação em Node, que inclui o reactNative, o React, e até mesmo angular e TypeScript que criemos componentes em arquivos separados, que são chamados em momentos de necessidade.

Este tipo de comportamento acontece visto que, em muitos sistemas, componentes acabam sendo muito parecidos e se repetindo com frequência como um padrão de botão ou caixa de texto, por isso, a partir de agora, vamos criar alguns componentes e montar nosso primeiro exemplo aplicável.

Começaremos criando uma nova hierarquia de pastas, contendo a pasta 'componentes' e dentro dela criaremos uma pasta para cada componente que queremos criar. Em nosso primeiro exemplo criaremos um botão customizado, logo o nome deste novo componente é **CustomButtom**. Para tal, vamos criar dois arquivos, um com o nome do nosso componente, "CustomButtom" e um chamado "Style" onde colocaremos o CSS deste componente.

Ficando com a seguinte hierarquia:



Vamos aos arquivos:

CustomButtom.js

```
import React from 'react';
import { TouchableOpacity, Text } from 'react-native';
import Styles from './Style'

function CustomButton({ title, onPress, style }) {
  return (
    <TouchableOpacity onPress={onPress} style={[Styles.button,
style]}>
      <Text style={Styles.buttonText}>{title}</Text>
    </TouchableOpacity>
  );
}

export default CustomButton;
```

A composição geral deste arquivo é semelhante ao primeiro que vimos.

Temos os ‘imports’ que trazem os conteúdos que precisamos, destacamos que o novo componente ‘TouchableOpacity’ que é a base de nosso botão, será explicado a frente e o componente Styles, que na verdade é o arquivos Styles que foi criado ao lado deste.

Já na função, nomeamos ela pelo nome que queremos chamar esse novo componente, que no nosso caso é a ‘CustomButton’. Esta função recebe parâmetros, logo quando ele for ser utilizado, deve-se passar tais parâmetros, que são ‘title’, ‘onPress’ e o ‘style’. Estes parâmetros serão utilizados na implementação do componente.

Agora vamos ao componente, o ‘TouchableOpacity’, este componente é um dos mais populares de botões presentes no ReactNative, sendo altamente customizável e muito fácil de implementar.

Vemos que o mesmo, implementa duas propriedades, o ‘onPress’, evento que ocorre ao ser apertado, e o ‘Style’, a folha de estilo do mesmo. Ao reparar nestas duas propriedades, podemos ver que as variáveis que vieram via parâmetro estão sendo utilizadas aqui, o evento a ser executado pelo ‘onPress’ será passado pela variável ‘onPress’ e o

estilo CSS que será executado também será passado via variável 'style', porém este tem uma peculiaridade, ele também acrescenta um estilo próprio, o 'Style.button' que será implementado no outro arquivo.

Já dentro do componente, um outro é criado, um <text> que permite um texto específico ser escrito dentro do botão, este texto também vem via parâmetro pela variável 'title' e o mesmo também possui um CSS que está sendo implementado diretamente pelo 'Styles.buttonText'.

Por fim, temos o 'export default CustomButton;' que serve para exportar este componente. Com esta exportação, o componente se torna acessível e implementável em outras páginas quando necessário.

Agora vamos para o arquivo da folha de estilo:

Style.js

```
import { StyleSheet } from 'react-native'

export default StyleSheet.create({

  button: {
    backgroundColor: '#1e90ff',
    paddingVertical: 12,
    paddingHorizontal: 25,
    borderRadius: 5,
    alignItems: 'center',
  },
  buttonText: {
    color: '#fff',
    fontSize: 16,
    fontWeight: 'bold',
  },
});
```

Neste arquivo, muito mais simples, temos apenas uma importação do componente da folha de estilo, como no exemplo anterior e já exportamos diretamente o resultado desta folha de estilo, onde criamos o componente como queremos através de comandos CSS, não é pretensão deste texto entrar em detalhes do CSS, porém podemos dizer

que deixará o botão arredondado, centralizado e verde, já o texto do botão ficará branco de fonte 16 e negrito.

Implementação

Agora vamos implementar o customButton na nossa pagina principal, ao abrir o projeto inicial, vamos customizar da seguinte forma:

```
import { StatusBar } from 'expo-status-bar';
import { StyleSheet, Text, View, Alert } from 'react-native';
import CustomButton from './componentes/CustomButton/CustomButton'

export default function App() {
  return (
    <View style={styles.container}>
      <CustomButton title="Press Me"
        onPress={() => Alert.alert('Button Pressed!')}
        style={{ backgroundColor: '#4caf50' }}
      />
      <StatusBar style="auto" />
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    alignItems: 'center',
    justifyContent: 'center',
  },
});
```

Este código é muito semelhante ao original, vamos destacar apenas as principais diferenças, na linha 2, onde importamos os componentes Text e View, adicionamos o Alert, um componente de caixa de mensagem para fazer uma caixa aparecer na tela quando o botão for clicado.

Na linha seguinte, a importação do customButton é feita, reparem bem no endereço do arquivo, deve estar correto para o componente ser encontrado.

Depois podemos ver a chamada do 'CustomButtom' como vemos abaixo:

```
<CustomButton title="Press Me"
  onPress={() => Alert.alert('Button Pressed!')}
  style={{ backgroundColor: '#4caf50' }}
/>
```

Basicamente temos a chamada do componente e os valores dos parâmetros que criamos no componente principal estão aqui.

Ao executar o programa temos o seguinte:

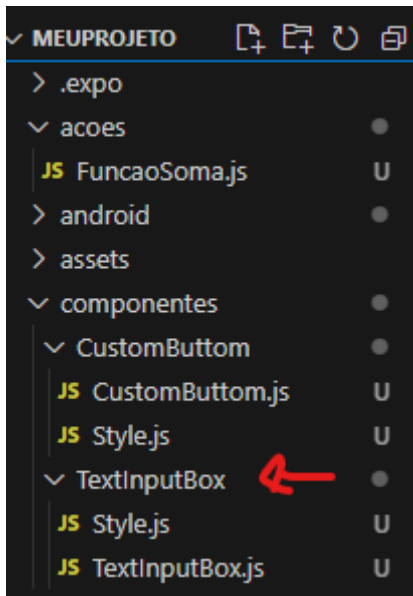


Press Me

Agora vamos criar o componente da caixa de texto e executar uma operação matemática:

Caixa de texto

A Caixa de texto é mais simples, vamos criar na pasta de componentes, uma nova pasta: a “TextInputBox” e dentro dela criaremos os arquivos “TextInputBox.js” e “Style.js”. Como mostra a imagem:



Para a programação desta “TextInputBox.js” faremos o seguinte:

```
import React from 'react';
import {TextInput} from 'react-native';
import Style from "./Style"

function TextInputBox({ value, onChangeText, placeholder,
keyboardType, style }) {
  return (
    <TextInput
      style={[Style.input, style]}
      value={value}
      onChangeText={onChangeText}
      placeholder={placeholder}
      keyboardType={keyboardType}
    />
  );
}

export default TextInputBox;
```

Não seremos redundantes na explicação, o funcionamento é basicamente o mesmo do botão que criamos anteriormente, com a

principal diferença sendo o uso do “TextInput” um componente padrão do React para caixas de texto.

Já o arquivo ‘Style.js’ segue a mesma receita:

```
import { StyleSheet } from 'react-native'

export default StyleSheet.create({

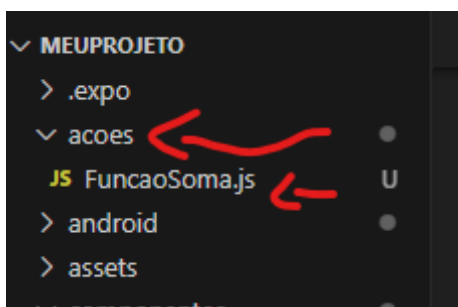
  input: {
    height: 40,
    borderColor: '#ccc',
    borderWidth: 1,
    borderRadius: 5,
    paddingHorizontal: 10,
    marginVertical: 10,
    width: '80%',
  },
});
```

Nada fora do que foi feito no botão, apenas uma folha de estilo simples.

Agora, vamos a página principal e ao processo de inserir lógica nesta tela

Tela principal e cálculo

Para tal, vamos iniciar criando um novo arquivo que irá contar a lógica deste processo. Vamos criar uma pasta chamada ‘acoes’ e um arquivo chamado ‘FuncaoSoma.js’



Vamos programar a “FuncaoSoma.js” com a logica que queremos executar em nosso processo.

```
import { Alert } from 'react-native';

export default function funcaoSoma(number1, number2) {
  const sum = parseFloat(number1) + parseFloat(number2);
  if (isNaN(sum)) {
    Alert.alert('Erro', 'Por favor, insira números válidos.');
```

Este trecho contém a lógica para somar dois números, não tem grandes detalhes neste código, visto que é apenas javascript puro e simples, porém vamos ressaltar alguns detalhes, como o IF que faz uma verificação de erro na variável, impedindo que haja algum erro de execução durante o cálculo, e, é claro, o import do 'Alert' necessário para a chamada da caixa na tela.

Para finalizar, vamos a pagina principal, o 'App.js' construir nossa aplicação:

```
import React, { useState } from 'react';
import { StatusBar } from 'expo-status-bar';
import { StyleSheet, Text, View } from 'react-native';
import CustomButton from '../componentes/CustomButton/CustomButton';
import TextInputBox from '../componentes/TextInputBox/TextInputBox';
import FuncaoSoma from '../acoes/FuncaoSoma';

export default function App() {
  const [number1, setNumber1] = useState("");
  const [number2, setNumber2] = useState("");

  return (
    <View style={styles.container}>
      <StatusBar style="auto" />

      <Text style={styles.title}>Soma de Dois Números</Text>
      <TextInputBox
        value={number1}
```

```

    onChangeText={setNumber1}
    placeholder="Digite o primeiro número"
    keyboardType="numeric"
  />
  <TextInputBox
    value={number2}
    onChangeText={setNumber2}
    placeholder="Digite o segundo número"
    keyboardType="numeric"
  />
  <CustomButton
    title="Somar"
    onPress={() => FuncaoSoma(number1,number2)}
    style={styles.button}
  />

</View>
);
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    alignItems: 'center',
    justifyContent: 'center',
  },
  title: {
    fontSize: 24,
    marginBottom: 20,
  }
});

```

A lógica por si só não é complexa, é apenas uma chamada de objetos e seus parâmetros sendo preenchidos, porém vamos analisar alguns pontos que trazem coisas úteis para nosso estudo:

- **Atenção aos imports:** É feito o import de todos os objetos, o que inclui o calculo que foi criado.
- **A Criação de variáveis:** O Processo de criação de uma variável é bem simples, deve-se utilizar este comando abaixo, o nome da

variável no nosso caso é `number1`, e de seu método de 'set' é 'setNumber1'. Ex: **`const [number1, setNumber1] = useState("");`**
Como estamos falando de javascript, não precisamos tipar.

- **useState**: Neste método, temos o 'useState', que cria a variável e o método de 'set' além de setar um valor inicial, em nosso exemplo é uma string vazia.

O resto funciona de forma padrão, ao executar o código temos o seguinte resultado:

Soma de Dois Números

Somar

Capítulo 3: Introdução ao React Native e Expo

NOME: _____ N° _____

TURMA: _____

PROFESSOR: _____

DATA: __/__/__

NOTA:

Exercícios

1 – O React Native possui uma estrutura de pastas padrão, em sua opinião por que devemos dividir os arquivos em pastas segmentadas?

2 – As aplicações React native possuem componentes nativos e customizados, diferencie cada um.

3 – Por que devemos utilizar o CSS como ferramenta dentro das nossas aplicações.

4 – o componente botão possui um evento padrão, qual seria esse? e qual o comando para usar?

5 – Qual o motivo para customizar os objetos de uma aplicação?

Atividade Prática

1 – Baseando-se no exemplo, crie um app que faça as 4 operações.

Capítulo 4: Componentes principais do React Native.

4.1 Introdução aos componentes

Analisando os capítulos anteriores, conhecemos alguns destes elementos, o Text ou o Label, TextInputBox e até o botão, TouchableOpacity, e através deles, criamos nossos componentes personalizados. Indo para este lado, vamos observar os componentes principais do ReactNative, e uma breve descrição de cada um desses componentes.

4.2 Componentes core

4.2.1 Exibindo conteúdo:

- **View:** A base para construção da UI. Um container flexível usado para agrupar elementos e aplicar estilos.
- **Text:** Exibe texto com estilos. Suporta nesting para formatação complexa.
- **Image:** Exibe imagens de diferentes fontes (local, web, etc.).
- **ScrollView:** Permite rolagem quando o conteúdo ultrapassa a tela.
- **FlatList/SectionList:** Exibem listas de dados de forma eficiente, com otimização de performance. FlatList para listas simples e SectionList para listas com seções.
- **Pressable:** Detecta toques e executa uma ação quando pressionado.
- **RefreshControl:** Permite atualizar o conteúdo com o gesto de puxar para baixo.

4.2.2 Inputs e Controles:

- **TextInput:** Campo de texto editável para entrada de dados pelo usuário.
- **Button:** Botão acionável para interações.
- **Switch:** Componente para alternar entre dois estados (ligado/desligado).
- **Slider:** Permite selecionar um valor dentro de um intervalo definido.
- **Picker:** Lista de opções para seleção.

4.2.3 Componentes Android:

- **DrawerLayoutAndroid**: Implementa o padrão de navegação "gaveta" do Android.
- **PermissionsAndroid**: Solicita permissões do usuário no Android.
- **ToastAndroid**: Exibe um pequeno aviso temporário ao usuário (Android).
- **BackHandler** (Android): Permite manipular o evento de pressionar o botão "Voltar" no Android.

4.2.4 Componentes iOS:

- **ActionSheetIOS**: Exibe um menu de opções em forma de folha (iOS).
- **AlertIOS**: Exibe um alerta com opções para o usuário (iOS).
- **DatePickerIOS**: Seletor de data para iOS.
- **ImagePickerIOS**: Permite selecionar imagens da biblioteca do dispositivo (iOS).
- **ProgressViewIOS**: Barra de progresso circular para iOS.
- **PushNotificationIOS**: Gerencia notificações push no iOS.
- **SegmentedControlIOS**: Controle segmentado para iOS.
- **Share**: Permite compartilhar conteúdo com outros aplicativos.
- **StatusBar**: Controla a barra de status do dispositivo (cor, visibilidade, etc.).

4.2.5 Componentes de UI:

- **ActivityIndicator**: Indicador de carregamento/atividade em andamento.
- **Modal**: Exibe conteúdo em uma janela flutuante sobreposta ao conteúdo principal.
- **KeyboardAvoidingView**: Ajusta automaticamente o layout para evitar que o teclado virtual cubra os campos de entrada.
- **VirtualizedList**: Renderiza apenas os itens visíveis na tela, melhorando a performance em listas grandes.

4.2.6 Outros:

- **SafeAreaView**: Renderiza o conteúdo dentro da área segura do dispositivo, evitando sobreposições com barras de status ou entalhes.
- **Dimensions**: Fornece informações sobre as dimensões da tela do dispositivo.
- **Platform**: Permite executar código específico para plataformas diferentes (iOS/Android).
- **Linking**: Abre URLs externas em navegadores ou outros aplicativos.
- **Animated**: API para criação de animações.
- **Gesture Responder System**: Sistema para lidar com gestos do usuário (toques, deslizar, etc.).
- **AccessibilityInfo**: fornece informações sobre o estado atual dos recursos de acessibilidade.
- **Appearance**: fornece a interface para consultar as preferências de aparência do usuário, por exemplo, o esquema de cores preferido (claro ou escuro).
- **BackHandler**: permite que você detecte quando o usuário pressiona o botão "Voltar" do hardware em seu dispositivo Android e, opcionalmente, executar algum código.
- **Clipboard**: fornece acesso à área de transferência do sistema, tanto para ler quanto para gravar.
- **InteractionManager**: ajuda a agendar a execução de animações e outras tarefas.

UFA!

São muitos componentes, e ainda existem as variações, como a de botão que utilizamos, o `TouchableOpacity`, que é amplamente utilizada pela comunidade e se encontra para uso de todos, é uma variação do `Button`.

Bom, para detalhes de todos os componentes, consulte o link ao lado.

(<https://reactnative.dev/docs/components-and-apis>)>>>

Vamos falar então pessoal dos principais componentes, e conforme os capítulos forem avançando, falaremos de mais alguns, conforme a necessidade.



Já aprendemos a colocar botões, textos, caixas de texto na tela, também conhecemos o componente Alert, o StyleSheet, que, apesar de não ser tangível, também é importante para nosso código.

4.2.7 Explorando.

Bom, para nosso próximo estudo, vamos falar de alguns componentes diferentes, partiremos do projeto da aula passada, o somador.

Em nosso código principal, o App.js temos:

```
import React, { useState } from 'react';
import { StatusBar } from 'expo-status-bar';
import { StyleSheet, Text, View } from 'react-native';
import CustomButton from './componentes/CustomButton/CustomButton';
import TextInputBox from './componentes/TextInputBox/TextInputBox';
import FuncaoSoma from './acoes/FuncaoSoma';

export default function App() {
  const [number1, setNumber1] = useState("");
  const [number2, setNumber2] = useState("");

  return (
    <View style={styles.container}>
      <StatusBar style="auto" />

      <Text style={styles.title}>Soma de Dois Números</Text>
      <TextInputBox
        value={number1}
        onChangeText={setNumber1}
        placeholder="Digite o primeiro número"
        keyboardType="numeric"
      />
      <TextInputBox
        value={number2}
        onChangeText={setNumber2}
        placeholder="Digite o segundo número"
        keyboardType="numeric"
      />
    </View>
  );
}
```

```

    <CustomButton
      title="Somar"
      onPress={() => FuncaoSoma(number1,number2)}
      style={styles.button}
    />

  </View>
);
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    alignItems: 'center',
    justifyContent: 'center',
  },
  title: {
    fontSize: 24,
    marginBottom: 20,
  }
});

```

Já neste código conhecido, podemos ver o componente 'View' e o componente 'StatusBar', começaremos falando da statusBar.

4.2.7.1 StatusBar

Este componente controla a barra superior do smartphone, ou seja onde ficam as notificações, a bateria, o relógio, etc. Ao utilizar a configuração que está ali no exemplo, style='auto' o efeito esperado é que a barra se comporte como o padrão do aparelho está configurado, geralmente mantém a cor do tema padrão ou da própria aplicação.

Vejamos outras configurações:

```

<StatusBar barStyle="dark-content" />, <StatusBar
barStyle="light-content" />

```

Deixa a barra no modo escuro ou claro, modificando o tema base.

`<StatusBar animated={false} />`, `<StatusBar animated={true} />`

Faz com que exista ou não exista a animação de 'fade' ao utilizar a barra status. Padrão true.



`<StatusBar hidden={true} />`, `<StatusBar hidden={false} />`

Faz com que a barra suma ou apareça. Padrão false.

`<StatusBar backgroundColor="red" />`

Altera a cor do fundo da barra para uma cor determinada, como no exemplo:

Soma de Dois Números

Somar

4.2.7.2 View

Vamos agora ao View, ele é o componente organizador, ou seja, basicamente é nossa area principal apresentada, se vai aparecer na tela de sua aplicação, deve estar dentro de view.

Apesar de não ter grandes configurações para ele, deve-se ter em mente que, qualquer CSS aplicado a ele, é aplicado ao app como um todo, ou seja, funciona como uma div principal ou o body do html.

Em nosso exemplo, percebemos que à um CSS aplicado nele:

```
container: {  
  flex: 1,  
  backgroundColor: '#fff',
```

```
alignItems: 'center',
justifyContent: 'center',
}
```

Neste CSS, vemos ali, por exemplo, o elemento 'flex' que configura o view para ocupar a totalidade da tela. Vemos mudança da cor de fundo, e do alinhamento de tudo que estiver lá dentro. Também podemos colocar coisas como 'padding' e até mesmo 'color', que mudaria a cor do texto de tudo que está escrito na tela que não possui cor definida.

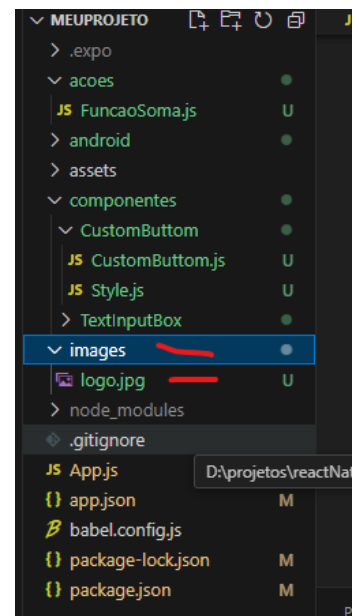
Agora vamos adicionar um componente.

4.2.7.3 Image

Seguimos, agora com o componente de imagem, ou melhor 'image', o nome é auto explicativo, aplica uma imagem na nossa aplicação.

Vamos começar colocando nossa imagem no projeto, para isso criaremos uma pasta no nosso projeto chamada 'images' e nele colocaremos uma imagem jpg. Veja como ficou na imagem ao lado.

Seguindo, agora que temos a imagem adicionada, podemos colocar a mesma no nosso projeto.



```
import React, { useState } from 'react';
import { StatusBar } from 'expo-status-bar';
import { StyleSheet, Text, View, Image } from 'react-native';
import CustomButton from
'./componentes/CustomButton/CustomButton';
import TextInputBox from './componentes/TextInputBox/TextInputBox';
import FuncaoSoma from './acoes/FuncaoSoma';

export default function App() {
  const [number1, setNumber1] = useState("");
  const [number2, setNumber2] = useState("");
```

```

return (
  <View style={styles.container}>
    <StatusBar style="auto" />

    <Image
      style={styles.logo}
      source={require('./images/logo.jpg')}
    />

    <Text style={styles.title}>Soma de Dois Números</Text>
    <TextInputBox
      value={number1}
      onChangeText={setNumber1}
      placeholder="Digite o primeiro número"
      keyboardType="numeric"
    />
    <TextInputBox
      value={number2}
      onChangeText={setNumber2}
      placeholder="Digite o segundo número"
      keyboardType="numeric"
    />
    <CustomButton
      title="Somar"
      onPress={() => FuncaoSoma(number1,number2)}
      style={styles.button}
    />

  </View>
);
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    alignItems: 'center',
    justifyContent: 'center',
  },
  title: {
    fontSize: 24,
    marginBottom: 20,

```

```
},  
  logo:{  
    width: 100,  
    height: 100,  
    marginBottom: 20  
  }  
});
```

Temos a alteração do nosso código, contendo agora o componente `image`. Nele podemos ver o seguinte:

```
import { StyleSheet, Text, View, Image } from 'react-native';  
...  
<Image  
  style={styles.logo}  
  source={require('./images/logo.jpg')}  
/>  
...  
logo:{  
  width: 100,  
  height: 100,  
  marginBottom: 20  
}
```

Vemos aqui em cima as linhas adicionadas, nelas, temos a primeira, que apenas adiciona a dependência. Depois temos a criação do componente em si, que adiciona a imagem que queremos no projeto, através da propriedade `'source'` e um CSS específico para ela, através do `'style'`.

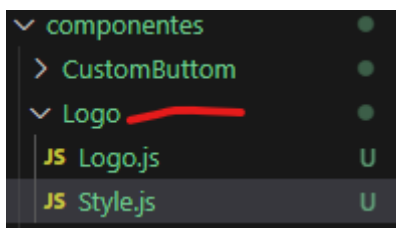
Por fim, temos um CSS próprio para ela, adicionando tamanho e margem especificadas.

Vamos refatorar isso.

4.2.7.3.2 Refatorando.

O que acontece, o logo de um projeto por exemplo, geralmente é utilizado com frequência em vários momentos do projeto, então não é legal deixar ele solto assim na página principal, vamos criar um componente customizado do 'image' para ele, como fizemos para o botão e a caixa de texto.

Começamos criando os componentes customizados no nosso projeto, como já fizemos antes:



No caso, criamos o componente logo. Vamos começar criando a folha de estilo:

```
import { StyleSheet } from 'react-native'
export default StyleSheet.create({
  logo:{
    width: 100,
    height: 100,
    marginBottom: 20
  }
});
```

Na prática, apenas trouxemos o que estava no App.js para o arquivo correto, com os devidos ajustes para exportar o resultado.

Agora vamos ao arquivo Logo.js:

```
import React from 'react';
import { Image } from 'react-native';
import Styles from './Style'

function Logo() {
```

```
return (  
  <Image  
    style={Styles.logo}  
    source={require('../images/logo.jpg')}  
  />  
);  
}  
export default Logo;
```

Basicamente o processo é o mesmo, trouxemos o objeto image para cá, nomeamos como 'Logo', e ajustamos para que o style fosse importado corretamente.

Com isso, nossa inclusão foi feita com sucesso. A Partir de agora, vamos sempre criar os componentes personalizados quando for necessário.

Antes de mostrar o código final do App.js, vamos observar um último componente para este capítulo.

4.2.7.4 Scrollview.

O componente ScrollView é focado em criar uma barra de rolagem para suas visualizações, ou seja, se porventura você colocar mais componentes em uma tela do que cabe, este objeto garante que uma barra de rolagem será criada ao lado do objeto.

Este componente possui várias configurações, porém também funciona sem nenhuma delas. Tais configurações podem ser encontradas:

<https://reactnative.dev/docs/scrollview>

Vamos ao código final do nosso projeto:

20:44

Bluetooth 4G Wi-Fi 86

teste

Soma de Dois Números

Digite o primeiro número

Digite o segundo número

Somar



```
import React, { useState } from 'react';
import { StatusBar } from 'expo-status-bar';
import { StyleSheet, Text, View, ScrollView } from 'react-native';
import CustomButton from './componentes/CustomButton/CustomButton';
import TextInputBox from './componentes/TextInputBox/TextInputBox';
import Logo from './componentes/Logo/Logo';
import FuncaoSoma from './acoes/FuncaoSoma';
```

```
export default function App() {
  const [number1, setNumber1] = useState("");
  const [number2, setNumber2] = useState("");

  return (
    <View style={styles.container}>
      <StatusBar style="auto" />
      <ScrollView contentContainerStyle={styles.container} >
        <Logo />
        <Text style={styles.title}>Soma de Dois Números</Text>
        <TextInputBox
          value={number1}
          onChangeText={setNumber1}
          placeholder="Digite o primeiro número"
          keyboardType="numeric"
        />
        <TextInputBox
          value={number2}
          onChangeText={setNumber2}
          placeholder="Digite o segundo número"
          keyboardType="numeric"
        />
        <CustomButton
          title="Somar"
          onPress={() => FuncaoSoma(number1,number2)}
          style={styles.button}
        />
      </ScrollView>
    </View>
  );
}
```

```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    alignItems: 'center',
    justifyContent: 'center',
  },
  title: {
    fontSize: 24,
    marginBottom: 20,
  },
  scroll:{
    backgroundColor: '#fff',
  }
});
```

Podemos ver no código a inserção do objeto customizado e do scrollView. A partir de agora, utilizaremos o ScrollView sempre que for necessário.

Capítulo 4: Componentes principais do React Native.

NOME: _____ N° _____

TURMA: _____

PROFESSOR: _____

DATA: __/__/__

NOTA:

Exercícios

- 1 – Defina com suas palavras a diferença entre componentes customizados e do core.
- 2 – Por que, durante o exemplo acima, se optou por utilizar um componente customizado na hora de fazer o logo do projeto?
- 3 – O componente Statusbar, possui uma série de configurações, qual seria necessária para deixar o modo escuro?
- 4 – (Descubra) Por que devemos adicionar 'novamente' o CSS no scrollView?
- 5 – O componente image, precisou de uma série de CSS, por que foi utilizado o width e o height?

Atividade Prática

1 – Crie um novo projeto, desta vez, contendo uma estrutura de decisão. O usuário deve digitar 3 notas e, após apresentar a média, deve-se dizer se nosso aluno foi aprovado ou reprovado. $\text{Media} = 6$. Não esqueça de colocar uma imagem e o scrollView.

Capítulo 5: TabNavigator

5.1 Introdução.

Agora vamos para navegação, entre janelas, existem algumas maneiras de fazer esse tipo de navegação, neste capítulo vamos falar da 'TabNavigator'.

Esta técnica envolve a criação de um menu inferior, basicamente um menu na parte de baixo da tela do dispositivo é criado, permitindo a navegação entre páginas.

5.2 Baixando as bibliotecas

Para começar, vamos começar adicionando a biblioteca necessária no ReactNative para criar tal objeto.

```
npm install @react-navigation/bottom-tabs
```

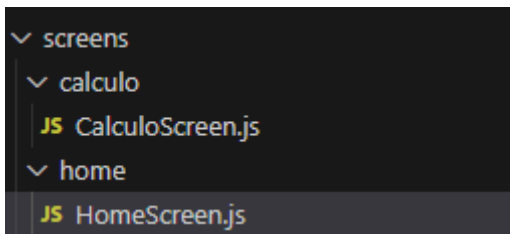
```
npm install @react-native-picker/picker
```

```
npm install @react-navigation/native
```

Com esta inclusão feita, vamos a criação das paginas novas.

5.3 Criando as paginas.

Vamos começar criando os arquivos. Veja abaixo a hierarquia que criamos:

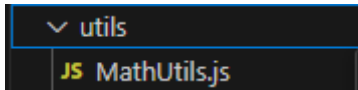


Na raiz do projeto, foi criado a pasta 'screens' e dentro dela uma pasta para cada página que queremos montar e dentro destas pastas os arquivos JS com o nome da página em questão, que no nosso caso é CalculoScreen.js e HomeScreen.js.

OBS: Não foi criado um CSS separado em cada página, visto que não fazia sentido para nosso projeto neste momento, deixando o css de cada

um dentro da própria página, porém fica claro que seria o ideal separar o CSS do arquivo base.

Para nossos arquivos auxiliares, vamos colocar arquivos que já foram explorados anteriormente, como o 'CustomButton' e o 'TextInputBox', 'Logo'. Já o cálculo colocaremos separado em uma nova função, criada em uma pasta chamada 'utils' sob o nome de 'MathUtils.js', como na imagem abaixo:



Vamos começar explorando este código em específico, sendo mais simples e rápido:

```
import { Alert } from 'react-native';

class MathUtils{
  static funcaoCalculo(number1, number2, acao) {
    var sum;
    switch (acao) {
      case '+':
        sum = parseFloat(number1) + parseFloat(number2);
        break;
      case '-':
        sum = parseFloat(number1) - parseFloat(number2);
        break;
      case '*':
        sum = parseFloat(number1) * parseFloat(number2);
        break;
      case '/':
        sum = parseFloat(number1) / parseFloat(number2);
        break;

      default:
        break;
    }
    if (isNaN(sum)) {
      Alert.alert('Erro', 'Por favor, insira números válidos.');
```

```
    Alert.alert('Resultado', `A soma é: ${sum}`);
  }
}
export default MathUtils;
```

Este código nada mais implementa um método que executa as quatro operações básicas. Nada fora do padrão já conhecido.

Agora vamos para a implementação da HomeScreen.

```
import * as React from 'react';
import { View, Text, Image, StyleSheet } from 'react-native';

function HomeScreen() {
  return (
    <View style={{ flex: 1, justifyContent: 'center', alignItems: 'center' }}>
      <Image
        style={styles.logo}
        source={require('../images/logo.jpg')}
      />
      <Text>Home Screen</Text>
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    alignItems: 'center',
    justifyContent: 'center',
  },
  title: {
    fontSize: 24,
    marginBottom: 20,
  },
});
```

```
picker: {
  height: 50,
  width: 200,
},
logo:{
  width: 100,
  height: 100,
  marginBottom: 20
}
});
export default HomeScreen;
```

Neste código não temos diferença também com os vistos anteriormente, visto que apenas implementamos uma view com uma imagem e um texto.

Agora vamos para o CalculoScreen:

```
import React, { useState } from 'react';
import { StatusBar } from 'expo-status-bar';
import { StyleSheet, View, Text } from 'react-native';
import TextInputBox from '../components/TextInputBox/TextInputBox';
import CustomButton from '../components/CustomButton/CustomButton';
import { Picker } from '@react-native-picker/picker';
import MathUtils from '../utils/MathUtils';

function CalculoScreen() {
  const [number1, setNumber1] = useState("");
  const [number2, setNumber2] = useState("");
  const [selectedValue, setSelectedValue] = useState('Somar');

  return (
    <View style={styles.container}>
      <StatusBar style="auto" />

      <Text style={styles.title}>Soma de Dois Números</Text>
      <TextInputBox
```

```

    value={number1}
    onChangeText={setNumber1}
    placeholder="Digite o primeiro número"
    keyboardType="numeric"
  />
  <Picker
    selectedValue={selectedValue}
    style={styles.picker}
    onChange={(itemValue, itemIndex) =>
setSelectedValue(itemValue)}
  >
    <Picker.Item label="Somar" value="+" />
    <Picker.Item label="Subtrair" value="-" />
    <Picker.Item label="Multiplicar" value="*" />
    <Picker.Item label="Dividir" value="/" />
  </Picker>

  <TextInputBox
    value={number2}
    onChangeText={setNumber2}
    placeholder="Digite o segundo número"
    keyboardType="numeric"
  />
  <CustomButton
    title="Calcular"
    onPress={() =>
MathUtils.funcaoCalculo(number1,number2,selectedValue)}
    style={styles.button}
  />

</View>
);
}

const styles = StyleSheet.create({
  container: {
    flex: 1,

```



```
    backgroundColor: '#fff',
    alignItems: 'center',
    justifyContent: 'center',
  },
  title: {
    fontSize: 24,
    marginBottom: 20,
  },
  picker: {
    height: 50,
    width: 200,
  }
});

export default CalculoScreen;
```

Vemos aqui um processo bem parecido com outros apresentados até o momento, criamos variáveis e utilizamos objetos de texto e também o objeto picker, que foi instalado anteriormente. Esta tela dá ao usuário a opção de digitar dois números e escolher qual operação ele deseja executar, e também por fim associa ao botão o evento de cálculo implementado pelo 'utils'.

5.4 aplicando o objeto TabControl.

Agora vamos criar o nosso App.js. Neste caso, temos que pensar que a página principal do nosso projeto apenas cuidará do menu, e as páginas que criamos que realmente carregarão o conteúdo.

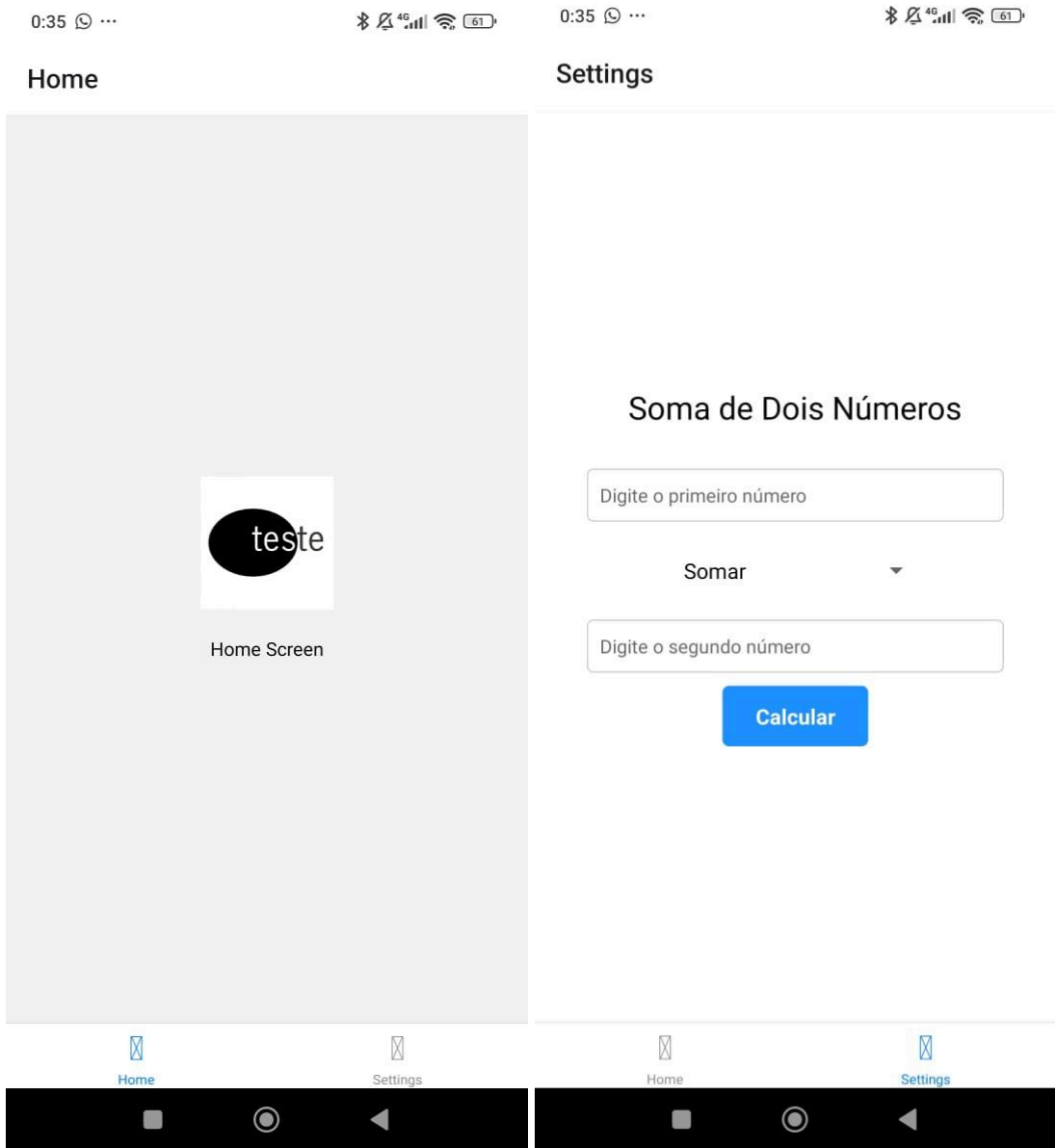
Vejamos:

```
import { NavigationContainer } from '@react-navigation/native';
import { createBottomTabNavigator } from '@react-navigation/bottom-tabs';
import CalculoScreen from './screens/calculo/CalculoScreen/';
import HomeScreen from './screens/home/HomeScreen/';

const Tab = createBottomTabNavigator();
```

```
export default function App() {  
  return (  
    <NavigationContainer>  
      <Tab.Navigator>  
        <Tab.Screen name="Home" component={HomeScreen} />  
        <Tab.Screen name="Settings" component={CalculoScreen} />  
      </Tab.Navigator>  
    </NavigationContainer>  
  );  
}
```

Funciona de forma bem simples, dentro do NavigatorContainer, colocamos o TabNavigator e por fim as Tab.Screens. Desta forma fica claro que, cada Tab.Screen contém uma das tabs que queremos criar. Em sua propriedade 'Name' colocamos o nome que queremos na página e na propriedade Component, colocamos o objeto instanciado da página que criamos. Vejamos o resultado:



Capítulo 5: TabNavigator

NOME: _____ N° _____

TURMA: _____

PROFESSOR: _____

DATA: __/__/__

NOTA:

Exercícios

- 1 – Descreva com suas palavras o funcionamento do componente View.
- 2 – Por que seria aconselhado separar o Style do arquivo JS das páginas?
- 3 – Qual razão de criarmos pastas individuais para cada tipo de coisa que criamos em nossa aplicação?
- 4 - Baseando-se no uso, por que não devemos criar muitas páginas nas Tabs?

Capítulo 6: Drawer e etc...

6.1 Introdução.

Agora vamos seguir nossa estrutura de telas para o Drawer.

O Drawer nada mais é do que o famoso menu sandwich, aquele que fica na lateral dos aplicativos e, ao clicar, um menu suspenso aparece.

Para nosso exemplo, partiremos do exemplo da aula anterior, facilitará bastante nosso processo.

Bom, vamos a implantação:

6.2 Baixando as bibliotecas

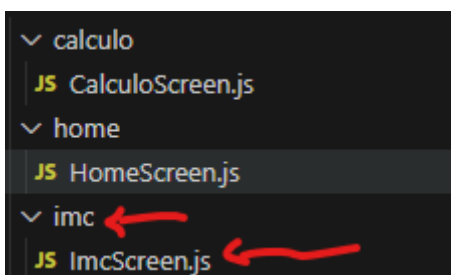
Para começar, vamos começar adicionando a biblioteca necessária no ReactNative para criar tal objeto.

```
npm install @react-navigation/drawer
```

Como partimos do projeto anterior, as bibliotecas anteriores também já estão preparadas.

6.3 Criando as páginas

Bom, vamos aproveitar o projeto anterior, logo nossa calculadora de 4 operações se encontra aqui, porém vamos criar uma nova página, a do IMC.



Criamos a pasta 'imc' na pasta de screens e dentro desta pasta o 'ImcScreen.js'.

Vamos ao código do ImcScreen:

```
import React, { useState } from 'react';
import { StatusBar } from 'expo-status-bar';
import { StyleSheet, View, Text } from 'react-native';
import TextInputBox from '../components/TextInputBox/TextInputBox';
import CustomButton from '../components/CustomButton/CustomButton';
import MathUtils from '../utils/MathUtils';
```

```

function CalculoScreen() {
  const [altura, setAltura] = useState("");
  const [peso, setPeso] = useState("");
  const [imc, setImc] = useState("");
  const [mensagem, setMensagem] = useState("");

  return (
    <View style={styles.container}>
      <StatusBar style="auto" />

      <Text style={styles.title}>IMC</Text>
      <TextInputBox
        value={altura}
        onChangeText={setAltura}
        placeholder="Digite sua Altura em cm"
        keyboardType="numeric"
      />

      <TextInputBox
        value={peso}
        onChangeText={setPeso}
        placeholder="Digite o peso em kg"
        keyboardType="numeric"
      />
      <CustomButton
        title="Calcular"
        onPress={() => MathUtils.calculaIMC(peso, altura, setImc, setMensagem)}
        style={styles.button}
      />

      <Text style={styles.mensagem}>{imc ? `Seu IMC é: ${imc}` : 'Insira seus dados para calcular o IMC'}</Text>
      <Text style={styles.mensagem}>{mensagem? mensagem : ""}</Text>

    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
  },
});

```

```

    alignItems: 'center',
    justifyContent: 'center',
  },
  title: {
    fontSize: 24,
    marginBottom: 20,
  },
  picker: {
    height: 50,
    width: 200,
  },
  mensagem: {
    fontSize: 18,
    marginBottom: 20,
  }
});

export default CalculoScreen;

```

Muito parecido com os códigos anteriores, neste cara, temos a entrada de peso e altura e o retorno do IMC.

O diferencial deste é que estamos retornando o número na tela, ou seja, no lugar do Alert, retornamos e preenchemos a tela com nosso cálculo.

Podemos ver isso na chamada da linha: “MathUtils.calculaIMC(peso,altura,setImc,setMensagem)” onde o método de cálculo, passa não só os dados necessários para o cálculo, mas também os métodos ‘set’ de duas variáveis de resposta, um contendo o número, e outra contendo uma mensagem que irá mostrar o índice do nosso cálculo. Estas variáveis serão preenchidas no nosso método utils, onde teremos o cálculo em si. Vamos a ele.

O método de cálculo que já existe em nosso projeto, será reutilizado aqui, segue o mesmo abaixo:

MathUtils.js

```

import { Alert } from 'react-native';

class MathUtils{

```



```

static funcaoCalculo(number1, number2, acao) {

    var sum;
    switch (acao) {
        case '+':
            sum = parseFloat(number1) + parseFloat(number2);
            break;
        case '-':
            sum = parseFloat(number1) - parseFloat(number2);
            break;
        case '*':
            sum = parseFloat(number1) * parseFloat(number2);
            break;
        case '/':
            sum = parseFloat(number1) / parseFloat(number2);
            break;

        default:
            break;
    }

    if (isNaN(sum)) {
        Alert.alert('Erro', 'Por favor, insira números válidos.');
```

```

    } else {
        Alert.alert('Resultado', `A soma é: ${sum}`);
    }
}

static calculaIMC(peso, altura, setImc, setMensagem){

    if (!altura || !peso) {
        setImc('Erro: Insira valores válidos');
        return;
    }
    altura = altura / 100;
    const imc = peso / (altura*altura);
    setImc(imc.toFixed(2));
    setMensagem(this.statusIMC(imc));
}

static statusIMC(imc){

```

```

    if(imc < 18.5){
      return "Abaixo do peso";
    }else if(imc < 24){
      return "Normal"
    }else if(imc < 30){
      return "SobrePeso grau 1";
    }else if(imc < 40){
      return "Obesidade grau 2"
    }else{
      return "Obsidade Grave grau 3";
    }
  }
}
export default MathUtils;

```

Foram criados dois métodos, o 'calculaIMC' e o 'statusIMC'. O 'calculaIMC' é o que é chamado pela nossa página de ação, nele foram passadas as variáveis contendo o peso e a altura e também os métodos 'set' da mensagem e do imc. Neste método é feito o cálculo e, utilizando o método 'setImc' o valor é colocado na tela. Já a mensagem, para ser preenchida, passa pela lógica do método 'statusIMC' que retorna o resultado esperado.

6.4 Implantando o menu.

Agora, o que nos resta é apenas a implementação do menu Drawer, ou seja, vamos mexer no App.js.

```

import { NavigationContainer } from '@react-navigation/native';
import { createDrawerNavigator } from '@react-navigation/drawer';
import CalculoScreen from './screens/calculo/CalculoScreen/';
import HomeScreen from './screens/home/HomeScreen/';
import ImcScreen from './screens/imc/ImcScreen/';

const Drawer = createDrawerNavigator();

export default function App() {
  return (
    <NavigationContainer>
      <Drawer.Navigator initialRouteName="Home">

```

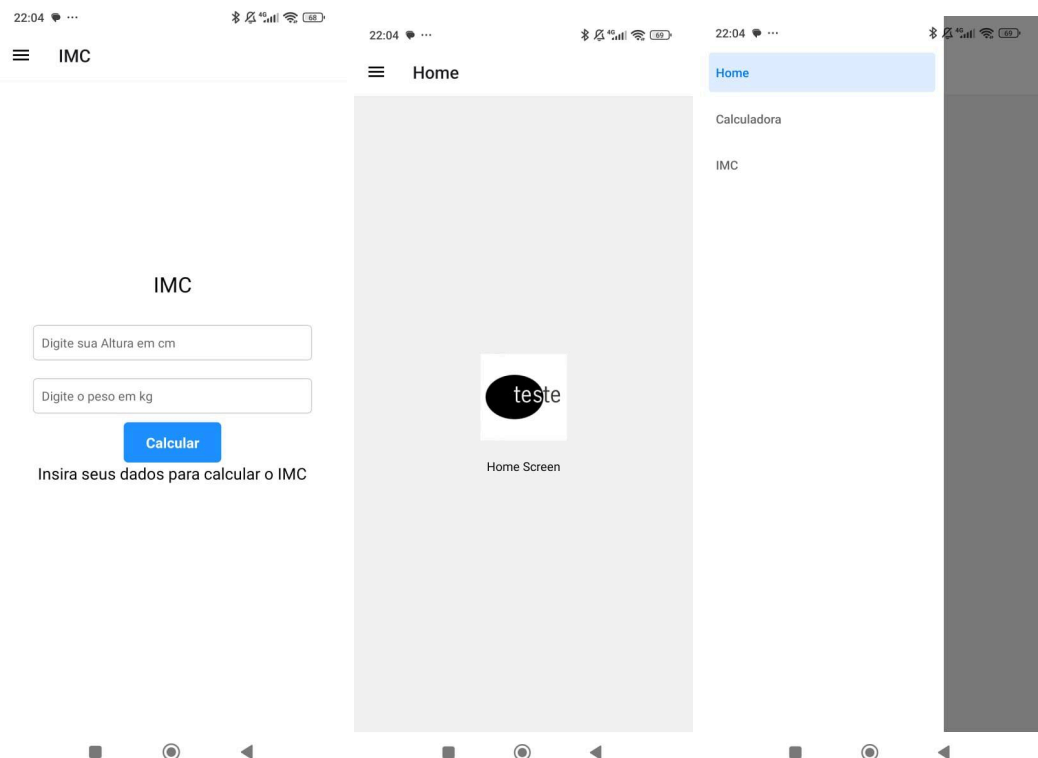
```

<Drawer.Screen name="Home" component={HomeScreen} />
<Drawer.Screen name="Calculadora" component={CalculoScreen} />
<Drawer.Screen name="IMC" component={ImcScreen} />
</Drawer.Navigator>
</NavigationContainer>
);
}

```

Repara-se bem na semelhança com o código anterior, substituindo apenas os “Tab.Screen” e ‘Tab.Navigator’ por suas versões, ‘Drawer’.

Vamos observar o resultado na nossa nova guia e como ficou o menu:



Capítulo 6: Drawer

NOME: _____ N° _____

TURMA: _____

PROFESSOR: _____

DATA: __/__/____

NOTA:

Exercícios

- 1 – Como você enxerga a diferença entre o Drawer e as Tabs?
- 2 – Você acha que podemos utilizar as Tabs e o Drawer no mesmo projeto? Ficaria interessante?
- 3 – No capítulo, adicionamos na tela o conteúdo de uma variável calculada, como isso aconteceu?
- 4 – Por que, quando precisamos que um dado volte a tela, devemos passar via parâmetro o 'set' de uma variável?

Atividade Prática

1 – Crie um novo projeto ou utilize o projeto do exemplo, adicionando uma nova aba para calcular a equação do segundo grau, Bhaskara.

Capítulo 7: Rodando no celular diretamente via Cabo.

7.1 Introdução.

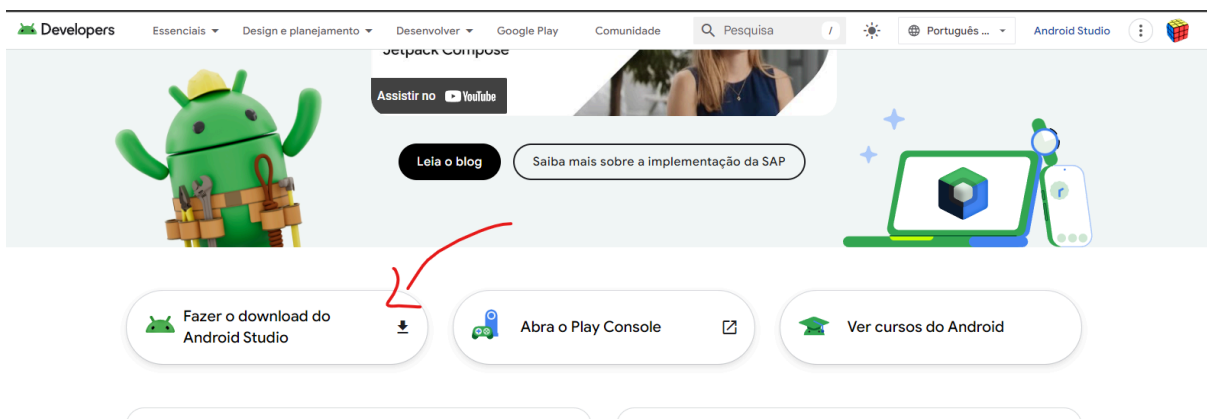
Até o momento, vínhamos executando o processo utilizando o expoGo e as simulações feitas através do navegador, porém vamos executar agora um fluxo que nos permitirá ter compilar em nossos celulares o fluxo e deixar o app rodando nativamente lá.

Faremos isso porque, devido a limitações do navegador, certas tarefas como banco de dados não são emulados nele, e em muitos casos não conseguimos utilizar o ExpoGo devido a necessidade de estarmos na mesma rede, uma limitação complicada em alguns casos de laboratórios e tals. Para isso, vamos emular diretamente no celular via cabo.

7.2 Instalação

Precisamos da instalação do Andoid Studio, devido a certas bibliotecas, como as DLLs do android e também as ferramentas que permitem a conexão entre o android e o computador, devemos ter esta ferramenta instalada na maquina. Vamos começar com ela então.

link: <https://developer.android.com/?hl=pt-br>



A instalação ocorre sem problemas, seguindo o velho esquema do Windows.

No geral, o Java ja deve estar instalado na máquina, mas caso não esteja, instale.

7.3 Preparando o SmartPhone.

Seguindo adiante, precisamos preparar o seu Smartphone para receber arquivos externos. No geral, por segurança, aplicações externas a lojas oficiais são barradas nos aplicativos, por isso vamos precisar acionar o 'modo desenvolvedor' de seu aparelho.

Para fazer isso, cada aparelho tem um modo diferente, mas no geral, dentro da área de configurações, vamos na guia de 'sobre o telefone' ou algo parecido.

Lá dentro localizamos alguns números, entre eles as 'Versão do OS' este número indica a versão do sistema operacional, devemos clicar nele de 5 a 10 vezes dependendo do modelo, clique lentamente que uma mensagem rápida vai aparecer na tela falando da quantidade de cliques que falta para liberar o modo desenvolvedor:



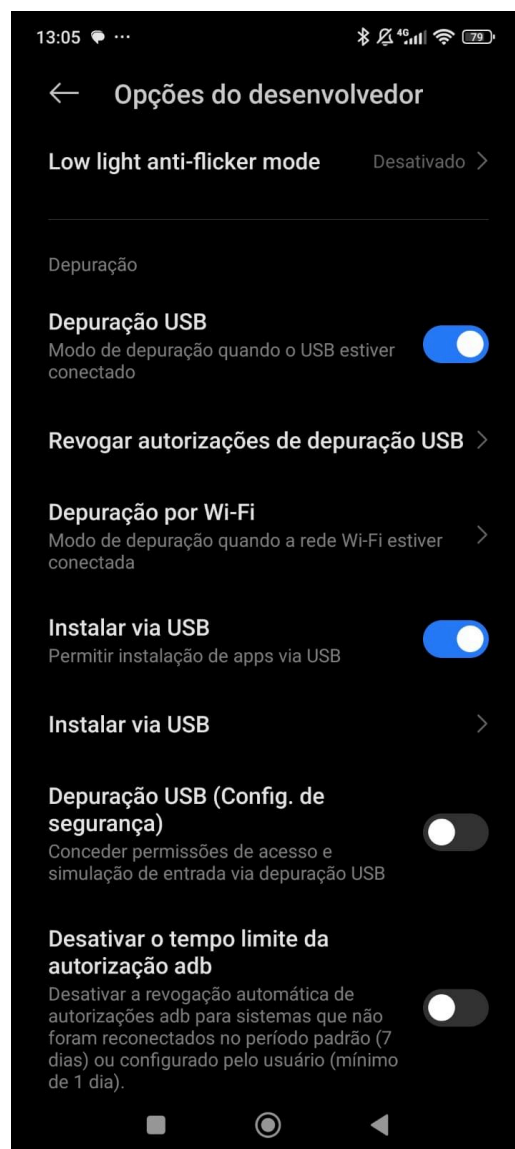
Ao clicar o número de vezes que você precisa, o modo desenvolvedor será liberado, dando acesso ao menu diretamente pelas configurações,

este menu 'surge' ele não está disponível antes de efetuar o procedimento acima.

Seguindo o fluxo, devemos entrar neste menu 'Opções do Desenvolvedor' ou algo parecido, recomendo efetuar a busca diretamente escrita ali.

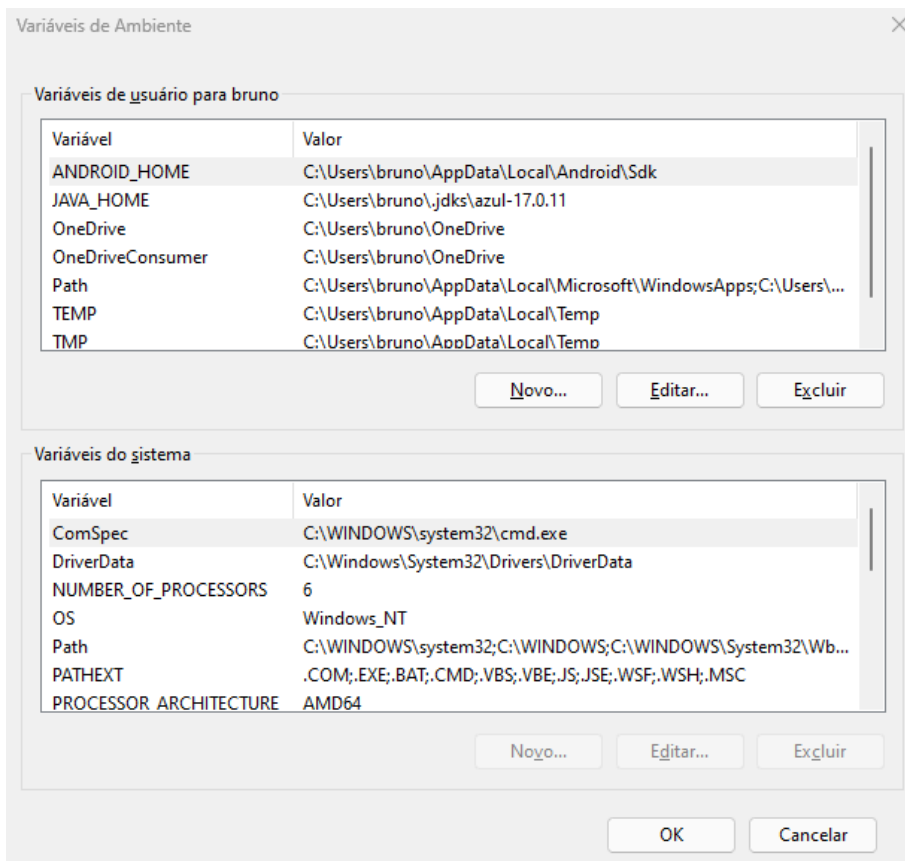
Neste menu, que é bem perigoso, não mexa levemente, vamos localizar a opção "instalar via USB" e "Depuração USB" e habilitar as mesmas.

OBS: Estas opções devem ser desativadas ao fim do uso, são perigosas e muitos aplicativos bancários e coisas semelhantes nem permitem ser abertos com elas habilitadas.

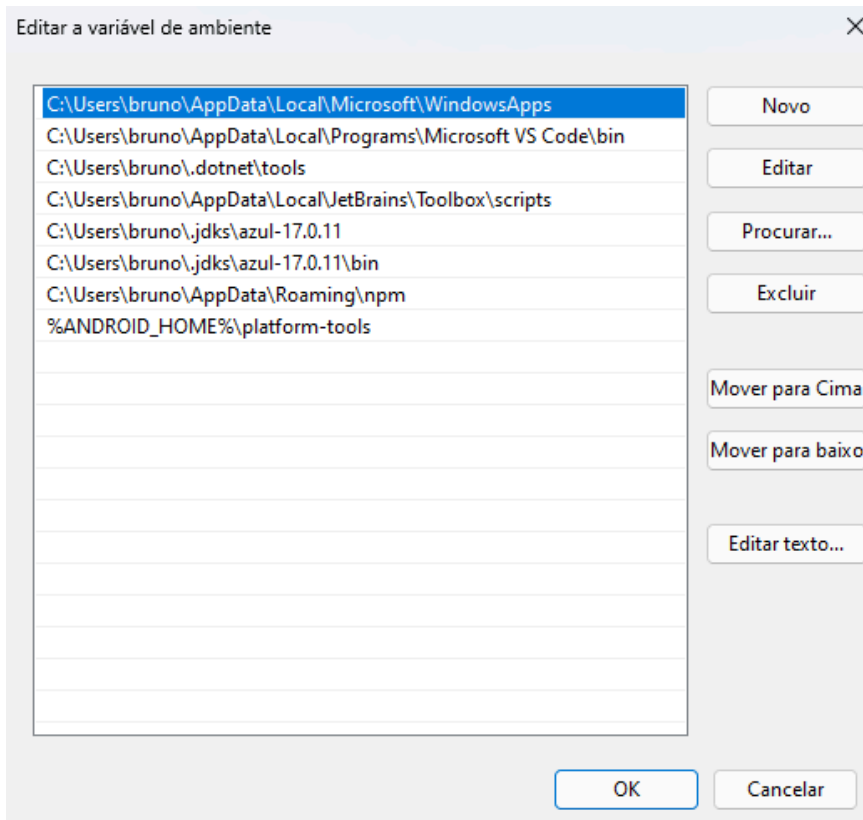


7.4 Ajustes das variáveis de ambiente.

Para dar sequência, precisamos garantir que as variáveis do windows estejam corretas, ou seja que esteja apontada para os locais corretos. Para validar isso, precisamos ir a area de variaveis de ambiente, clicando no menu 'Iniciar' e buscando por 'variáveis de ambiente' chegando na seguinte tela:



São necessárias algumas validações nas variáveis de ambiente do usuário, vejamos, devemos ter a variável 'JAVA_HOME' apontando para o java instalado na máquina, temos que ter a variável 'ANDROID_HOME' apontando para o SDK da instalação do Android, como vemos ali em cima, e por fim temos a variável Path, quando clicamos nele abre uma janela com várias linhas:



Vemos que precisamos das linhas referentes ao Java, como vimos ali e também a do android, apontando para as Tools das janelas. Com isso podemos seguir.

7.5 Aplicando a instalação no aparelho.

Agora vamos seguir para o momento derradeiro, a instalação do app.

Começamos plugando o cabo em nosso computador e em nosso aparelho.

A partir deste ponto, fique atento a tela de seu telefone, ele apresentará, esporadicamente, pedidos de permissão que devem ser aprovados.

Vamos para nosso terminal de controle, o mesmo em que rodamos a execução dos nossos testes, e de nosso servidor local.

Começamos com o primeiro comando:

```
adb devices -l
```

Este código retornará todos os celulares plugados em seu computador, como podemos ver a imagem abaixo:

```
PS D:\projetos\reactNative\menuSandwich> adb devices -l
* daemon not running; starting now at tcp:5037
* daemon started successfully
List of devices attached
82dff7f7          device product:tapas_global model:23021RAAEG device:tapas transport_id:1
```

Podemos ver aqui que um telefone foi retornado, o que precisamos desta etapa é o código, no meu caso é 23021RAAEG, este código será utilizado mais a frente. ANOTE.

Na sequencia, vamos compilar, para isso precisamos executar os seguintes comandos:

```
npx expo install expo-system-ui
```

```
npx expo prebuild
```

O resultado das execuções fica:

```
PS D:\projetos\reactNative\menuSandwich> npx expo install expo-system-ui
> Installing 1 SDK 51.0.0 compatible native module using npm
> npm install

added 1 package, changed 1 package, and audited 1244 packages in 7s

131 packages are looking for funding
  run `npm fund` for details

4 moderate severity vulnerabilities

To address issues that do not require attention, run:
  npm audit fix

To address all issues (including breaking changes), run:
  npm audit fix --force

Run `npm audit` for details.
```

```
PS D:\projetos\reactNative\menuSandwich> npx expo prebuild

 Android package Learn more: https://expo.fyi/android-package

✓ What would you like your Android package name to be? ... com.anonymous.menusandwich
✓ Created native directory
✓ Updated package.json
✓ Finished prebuild
```

Atenção!!

Agora vamos enviar o código:

```
npx expo run:android --device '23021RAAEG'
```

Repare, o código que foi descoberto anteriormente está presente aqui, para que o envio seja feito.

Este processo pode demorar alguns minutos, até dez minutos dependendo da situação.

Esta execução pode ocorrer erros, se acontecer, devido a alguma peculiaridade da compilação, você deve ler o erro e resolver a pendência em questão.

IMPORTANTE.

Ao fim do comando sendo executado, o celular irá pedir permissão novamente para instalar o app, e existe um tempo limite para responder, fique atento.

Capítulo 7: Rodando no celular diretamente via Cabo.

NOME: _____ N° _____

TURMA: _____

PROFESSOR: _____

DATA: __/__/__

NOTA:

Exercícios

- 1 – Por que é necessário que testes sejam feitos em celulares nativamente?
- 2 – O que seria fazer um teste nativo?
- 3 – O procedimento é grande e complexo, por que precisamos do Android Studio instalado para isso?
- 4 – Por que devemos ter cuidado ao liberar o modo desenvolvedor em um smartphone?
- 5 – Qual o objetivo nosso em liberar o modo desenvolvedor no smartphone?

Atividade Prática

1 – Conseguir fazer a compilação e executar em seu SmartPhone uma aplicação feita em aula.

Capítulo 8: Banco de dados - SQLite

8.1 Introdução.

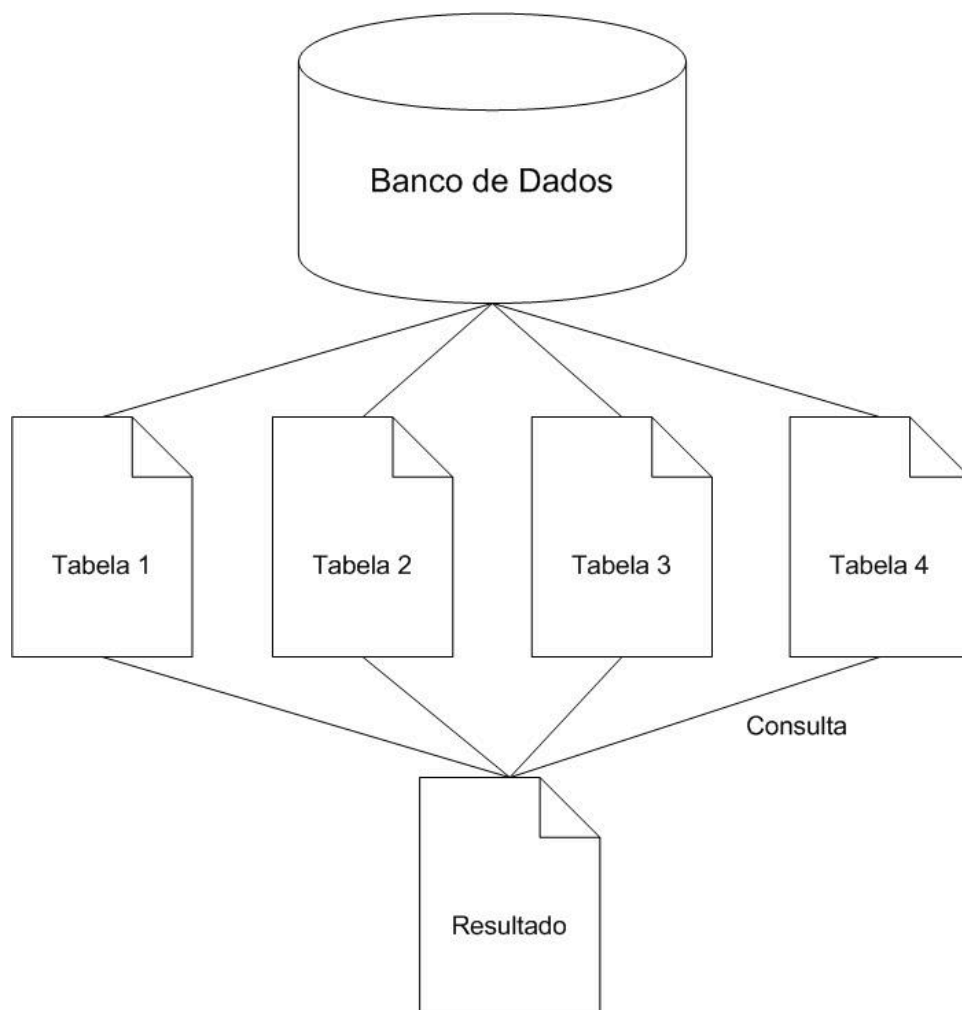
Agora vamos ao famigerado banco de dados. Neste ponto, o que queremos é que, em nossa situação, os dados sejam gravados e possamos utilizá-los quando necessário.

Bancos de dados são uma das grandes bases dos projetos e sem eles nossa vida seria muito diferente.

8.2 Entendendo bancos de dados.

Podemos definir banco de dados como uma forma de armazenar um conjunto de dados quaisquer, viabilizando a consulta de muitas maneiras e o controle geral destes dados.

Estas tecnologias são múltiplas, possuindo vários caminhos diferentes que podemos seguir. Faremos uma abordagem clássica dos bancos de dados, os bancos relacionais.



Bancos de dados relacionais são basicamente conjuntos de tabelas que, de uma forma ou de outra, armazenam os dados que queremos e nos fornece os mesmos de volta quando precisamos.

No geral, estes tipos de banco de dados utilizam SGBD's e a linguagem SQL para controlar seus fluxos.

8.3 SGBD

Ao programar de aplicativos para computadores sofreu profundas mudanças desde seu início, quando se usavam linguagens como Basic, Clipper, COBOL, C e outras. Os programadores incorporaram em um programa toda funcionalidade desejada. O programa continha as operações da interface de usuário, os cálculos, as operações de armazenamento de dados, bem como as tarefas de comunicação com outros sistemas.

Com o tempo, foram sendo identificadas funcionalidades comuns a muitos programas. Por exemplo, hoje em dia, não é necessário que se faça o cálculo dos dados em seu sistema, podendo ser feito pelo banco de dados e enviado ao sistema apenas a resposta destes dados.

Entretanto, normalmente o programa não contém todo código referente à exibição dos dados na interface, mas utilizam gerenciadores de interface de usuário, conjuntos de rotinas que incluem as funcionalidades que um programador vai necessitar frequentemente, ao construir uma interface de usuário. Da mesma forma, para comunicar-se com processos remotos, usam gerenciadores de comunicação. Para manter grandes repositórios compartilhados de dados, ou seja, para manter bancos de dados, são usados sistemas de gerência de banco de dados (SGBD).

Em resumo, estes SGBD são códigos prontos incorporados no banco de dados que permite que se utilize em qualquer programa feito em qualquer linguagem compatível.

8.4 SQL

SQL (Structured Query Language) ou linguagem de consulta estruturada é uma linguagem utilizada para comunicação entre software e bancos de dados relacionais declarada padrão no mercado.

Esta linguagem vem de meados de 1970 pela IBM, em uma época em que bancos de dados relacionais eram apenas projetos.

Esta linguagem é um grande padrão no mercado, devido a ser extremamente simples e de fácil entendimento. Sendo assim, ela possui grandes adeptos e com o tempo todas as grandes plataformas de Bancos de dados adotaram seus preceitos de funcionamento.



Após sofrer várias revisões ao decorrer de sua existência, hoje seus padrões são geridos pela ISO (International Organization for Standardization), facilitando muito sua padronização.

O SQL possui uma vantagem muito grande, quase todos os sistemas de bancos de dados que existem conseguem trabalhar com ele sem muitas diferenças entre si, ou seja, muito pouco é mudado e adaptado para cada sistema.

Suponhamos que temos um código escrito para o Oracle da empresa Oracle Corporation, e queremos que rode em um SQL Server, da Microsoft, com poucas adaptações é possível se fazer entender entre eles.

OBS: O nome do SQL originalmente era SEQUEL e por isso até hoje a sigla em inglês é pronunciada SÍ-QUE-EL.

Para as aplicações mobile, quando se precisa de um banco de dados relacional local, rodando diretamente no aparelho, utiliza-se o SQLite.

4.5 SQLite

Ao contrário de sistemas robustos como MySQL ou PostgreSQL, que exigem um servidor dedicado, o SQLite se diferencia por ser um banco de dados embarcado. Isso significa que sua biblioteca é integrada



diretamente ao aplicativo, eliminando a necessidade de um processo servidor separado. Imagine o SQLite como um "mini banco de dados" que reside dentro do próprio programa, proporcionando uma gestão de dados eficiente e independente.

A natureza compacta e independente do SQLite o torna ideal para uma gama diversificada de aplicações, especialmente em cenários onde a simplicidade e a portabilidade são primordiais. Alguns exemplos notáveis incluem:

- **Dispositivos Móveis:** Smartphones e tablets se beneficiam da leveza do SQLite para armazenar dados de aplicativos, históricos de navegação, contatos e muito mais.
- **Aplicações Desktop:** Softwares de todos os tipos, de editores de texto a players de música, utilizam o SQLite para gerenciar configurações, armazenar dados de usuário e garantir um funcionamento fluido.
- **Sistemas Embarcados:** Devido ao seu baixo consumo de recursos, o SQLite se torna uma escolha natural para dispositivos com poder de processamento limitado, como sensores, dispositivos IoT e sistemas de automação.
- **Armazenamento de Dados de Aplicativos:** Diversos softwares, mesmo aqueles que utilizam bancos de dados maiores como

backend, se beneficiam do SQLite para armazenar dados de configuração, cache e outras informações internas.

- **Prototipagem e Desenvolvimento:** A facilidade de uso e configuração rápida fazem do SQLite a escolha ideal para prototipagem e desenvolvimento de aplicações, permitindo que os desenvolvedores se concentrem na lógica do programa sem se preocupar com a complexidade da administração de um banco de dados.

O SQLite se destaca em diversas áreas, oferecendo um conjunto de vantagens significativas:

- **Facilidade de Implementação:** A curva de aprendizado do SQLite é extremamente suave, permitindo que mesmo desenvolvedores iniciantes o dominem rapidamente.
- **Manutenção Simplificada:** A administração do SQLite se resume a gerenciar um único arquivo, dispensando a necessidade de configurações complexas de servidor e backups elaborados.
- **Portabilidade Excepcional:** A capacidade de funcionar em diversos sistemas operacionais sem modificações no código é um dos grandes trunfos do SQLite.
- **Custo Zero:** Ser de domínio público garante que o SQLite seja completamente gratuito, sem custos de licença ou taxas adicionais.

Apesar de suas inúmeras vantagens, o SQLite possui limitações que devem ser cuidadosamente analisadas:

- **Escalabilidade Limitada:** O desempenho do SQLite pode ser afetado em cenários com grande volume de dados e alta concorrência de escrita.
- **Funcionalidades Restritas:** Comparado a bancos de dados cliente-servidor, o SQLite oferece um conjunto de recursos e funcionalidades mais limitado.
- **Segurança Relativa:** Como o banco de dados é armazenado em um único arquivo, a segurança depende da proteção do próprio arquivo. É crucial implementar medidas de segurança adicionais, especialmente em ambientes compartilhados.

Capítulo 8: Banco de dados - SQLite

NOME: _____ N° _____

TURMA: _____

PROFESSOR: _____

DATA: __/__/____

NOTA:

Exercícios

- 1 – Por que é necessário o uso de bancos de dados em programação? diga com suas palavras.
- 2 – O SQL é uma linguagem própria para bancos de dados, todos utilizam? por que?
- 3 – Defina com suas palavras o por que de existir vários SGBD's diferentes no mercado.
- 4 – SQLite é apenas utilizado em programação móvel? Por que?
- 5 – Sabendo das vantagens e desvantagens do SQLite, por que o mesmo é utilizado em vários cenários da programação mobile?

Capítulo 9: Banco de dados, aplicando nosso primeiro exemplo.

9.1 Introdução.

Neste momento, vamos começar a implementar o SQLite no nosso projeto em desenvolvimento.

Para facilitar nosso dia a dia, vamos utilizar a estrutura que já possuímos de outros exemplos para fluir nosso código.

OBS: Os experimentos foram feitos e testados para rodar diretamente no celular, o banco de dados geralmente não funciona bem emulado em ambiente web.

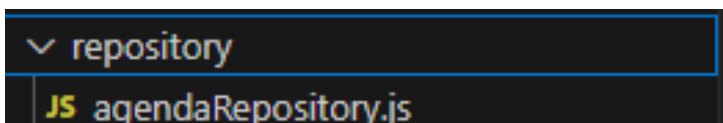
9.2 Adição das bibliotecas.

A seguinte biblioteca deve ser baixada viabilizar o sqlite no projeto, as outras já foram baixadas em momentos anteriores do exemplo:

```
npm install expo-sqlite
```

9.3 Modificando o código.

Vamos começar com a parte principal, a criação do arquivo de banco de dados. Para tal, vamos criar uma nova pasta chamada 'repository' e dentro dela um arquivo chamado 'agendaRepository.js', como podemos ver na imagem abaixo:



Neste arquivo, colocaremos os códigos referentes ao banco de dados, vamos começar colocando o código abaixo e depois explicaremos os pontos relevantes:

```
import * as SQLite from 'expo-sqlite/legacy';

const db = SQLite.openDatabase('agenda.db');

const createTable = () => {
  db.transaction(tx => {
    tx.executeSql(
```



```

    "CREATE TABLE IF NOT EXISTS agenda (id INTEGER
PRIMARY KEY AUTOINCREMENT, nome TEXT, telefone TEXT);",
    [],
    () => console.log("Tabela criada com sucesso."),
    (tx, error) => {
        console.log("Erro ao criar tabela:", error);
        return true;
    }
    );
    });
};

const getAgenda = (setAgenda) => {
    db.transaction(tx => {
        tx.executeSql(
            'SELECT * FROM agenda;',
            [],
            (_, { rows: { _array } }) => {
                console.log("Dados recuperados:", _array);
                setAgenda(_array);
            },
            (tx, error) => {
                console.log("Erro ao recuperar dados:", error);
                return true;
            }
        );
    });
};

const adicionarAgenda = (nome, telefone, setAgenda) => {
    db.transaction(tx => {
        tx.executeSql(
            'INSERT INTO agenda (nome, telefone) VALUES (?, ?);',
            [nome, telefone],
            () => {
                console.log("Dados inseridos com sucesso.");
                getAgenda(setAgenda);
            },
            (tx, error) => {
                console.log("Erro ao inserir dados:", error);
                return true;
            }
        );
    });
};

```

```

    );
  });
};

createTable();

export { db, getAgenda, adicionarAgenda, removerAgenda };

```

Muito bem, neste código, logo no início temos a criação do banco de dados `const db = SQLite.openDatabase('agenda.db');` este comando inicia o arquivo `'agenda.db'` e o cria caso o mesmo não exista.

Em seguida temos a criação de três métodos (funções), as `'const'`. Estas três têm funções distintas, porém tem seu código muito parecido. As três são: `'createTable'`, `'getAgenda'`, `'adicionarAgenda'`. Os nomes são auto-explicativos e já estudaremos o seu código. Por fim, ao fim do arquivo, temos a chamada do método `'createTable'` e o export de tudo que foi feito dentro deste arquivo.

Diferente dos arquivos que estudamos até agora, esta está exportando vários objetos de uma vez, visto que foram criados vários.

Vamos aos métodos, todos os três utilizam a função de execução no banco de dados:

```

db.transaction(tx => {
  tx.executeSql(
    'codigoSQL',
    [], //parametros
    () => {}, // o que fazer se der certo
    (tx, error) => {} //o que fazer se der errado.
  );
});
};

```

A função em si é bem simples, visto que a mesma apenas executa um SQL quando solicitada e, em seguida, toma uma ação diferente quando obtém sucesso ou quando algum problema ocorre.

9.3.1 createTable

Vejamos no nosso exemplo, no 'createTable', é executado um SQL que cria uma nova tabela caso a mesma não exista: "CREATE TABLE IF NOT EXISTS agenda (id INTEGER PRIMARY KEY AUTOINCREMENT, nome TEXT, telefone TEXT);"

Este comando é executado e o mesmo apresenta um sucesso ou um fracasso, em qualquer um dos casos, é utilizado um console.log para denotar que obtivemos sucesso ou fracasso nele.

9.3.2 getAgenda

No método de consulta, o 'getAgenda' é feito uma consulta simples, porém o método em questão recebe um 'setAgenda' como parâmetro, que na prática é uma variável para absorver o resultado desta consulta.

Neste caso, podemos ver que, caso obtenhamos sucesso na consulta, além da mensagem, os resultados da consulta são inseridos na variável 'getAgenda' para assim serem utilizados pelo front.

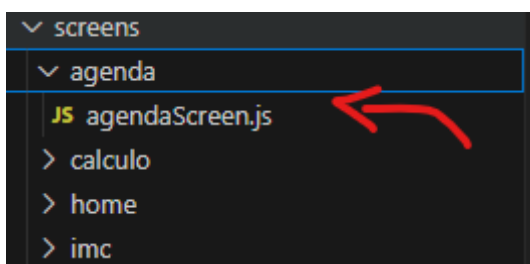
9.3.3 Finalizando código do banco.

Podemos perceber que o funcionamento é bem simples e que, com conhecimento em SQL, qualquer consulta é possível.

Ao fim do código, temos uma chamada seca do 'createTable', esta chamada ocorre porque, ao se chamar a biblioteca que estamos criando, eu já quero criar a tabela, assim garantindo que a mesma está criada sempre que for chamada.

9.3.4 Front.

Bom esta foi a parte mais difícil, agora vamos criar um front para interagir com nosso banco, para isso vamos na pasta 'screens' e criaremos a pasta 'agenda' e o arquivo 'agendaScreen.js'



Dentro deste arquivo, iremos colocar o conteúdo necessário para exibir e inserir novos registros em uma página. Vamos dar uma olhada no código e discutir o mesmo a seguir:

```
import React, { useState, useEffect } from 'react';
import { View, Text, TextInput, Button, FlatList, StyleSheet } from 'react-native';
import { getAgenda, adicionarAgenda, removerAgenda } from
'../../repository/agendaRepository';

export default function AgendaScreen() {
  const [novoNome, setNovoNome] = useState("");
  const [novoTelefone, setNovoTelefone] = useState("");
  const [agenda, setAgenda] = useState([]);

  useEffect(() => {
    getAgenda(setAgenda);
  }, []);

  return (
    <View style={styles.container}>
      <Text style={styles.titulo}>Agenda</Text>
      <View style={styles.inputContainer}>
        <TextInput
          style={styles.input}
          placeholder="Digite Nome"
          value={novoNome}
          onChangeText={setNovoNome}
        />
        <TextInput
          style={styles.input}
          placeholder="Telefone"
          value={novoTelefone}
          onChangeText={setNovoTelefone}
        />
        <Button title="Adicionar" onPress={() => {
          adicionarAgenda(novoNome, novoNovoTelefone, setAgenda);
          setNovoNome("");
          setNovoTelefone("");
        }}
        />
      </View>
      <Text style={styles.titulo}>LISTA:</Text>
```

```

    <FlatList
      data={agenda}
      keyExtractor={({item}) => item.id.toString()}
      renderItem={({item}) => (
        <View style={styles.item}>
          <Text>{item.nome}</Text>
          <Text> --- </Text>
          <Text>{item.telefone}</Text>
        </View>
      )}
    />
  </View>
);
}

```

```

const styles = StyleSheet.create({
  container: {
    flex: 1,
    padding: 20,
  },
  titulo: {
    fontSize: 24,
    fontWeight: 'bold',
    marginBottom: 20,
  },
  inputContainer: {
    flexDirection: 'row',
    marginBottom: 20,
  },
  input: {
    flex: 1,
    borderWidth: 1,
    borderColor: '#ccc',
    padding: 10,
    marginRight: 10,
  },
  item: {
    flexDirection: 'row',
    alignItems: 'center',
    padding: 10,
    borderBottomWidth: 1,
    borderBottomColor: '#ccc',
  },
});

```

```
},  
});
```

Não vamos nos ater aos detalhes deste código que já foram explicados em outros capítulos, o que precisamos ver aqui são as relações com o banco de dados.

Logo no início, percebemos que o mesmo está importando o arquivo de repository que acabamos de criar e em seguida cria as variáveis, 'novoNome', 'novoTelefone' e 'agenda'. As primeiras duas variáveis são auxiliares para obter dados novos a serem inseridos, já a terceira, conterá a lista do que já existe no banco. Podemos ver o preenchimento do dela logo em seguida com a chamada do 'useEffect'. Esta função basicamente chama uma função, no nosso caso, está chamando a 'getAgenda' que criamos com a consulta do banco, e utilizando o setAgenda, que criamos logo acima para preencher os dados na variável.

Em seguida, seguimos com um código normal, como já vimos outros, com objetos view e etc. No primeiro view, temos a criação de um pequeno formulário, com um botão para chamar a gravação do banco, o 'adicionarAgenda'.

Já no segundo view, temos a criação de uma lista (FlatList) que irá conter o valor armazenado dentro da variável agenda, sabemos que tal variável é um array, logo precisamos de uma lista para exibir seu conteúdo.

O objeto flatList, listado aqui, possui três parâmetros interessantes que merecem destaque.

- **data:** Este parâmetro é para setar a variável que contém os valores que queremos listar.
- **keyExtractor:** Como estamos falando de uma lista, precisamos de uma variável guia, que identifique cada item da lista individualmente, no nosso exemplo ficou o 'id' da tabela.
- **renderItem:** Este basicamente mostra como queremos montar cada linha de nossa lista, no nosso exemplo é feita uma repetição de cada item da variável 'item' e depois criamos os objetos text para exibir o nome e o telefone listados.

A construção não é tão complexa, e é bastante flexível, no fim das contas pode ser usada de várias formas diferentes.

9.3.5 Menu

Agora vamos finalizar, adicionando o novo item ao menu, para isso vamos ao 'App.js' principal e adicionaremos mais um item, vejamos como o código ficou:

```
import { NavigationContainer } from '@react-navigation/native';
import { createDrawerNavigator } from '@react-navigation/drawer';
import CalculoScreen from './screens/calculo/CalculoScreen/';
import HomeScreen from './screens/home/HomeScreen/';
import ImcScreen from './screens/imc/ImcScreen/';
import AgendaScreen from './screens/agenda/agendaScreen/';

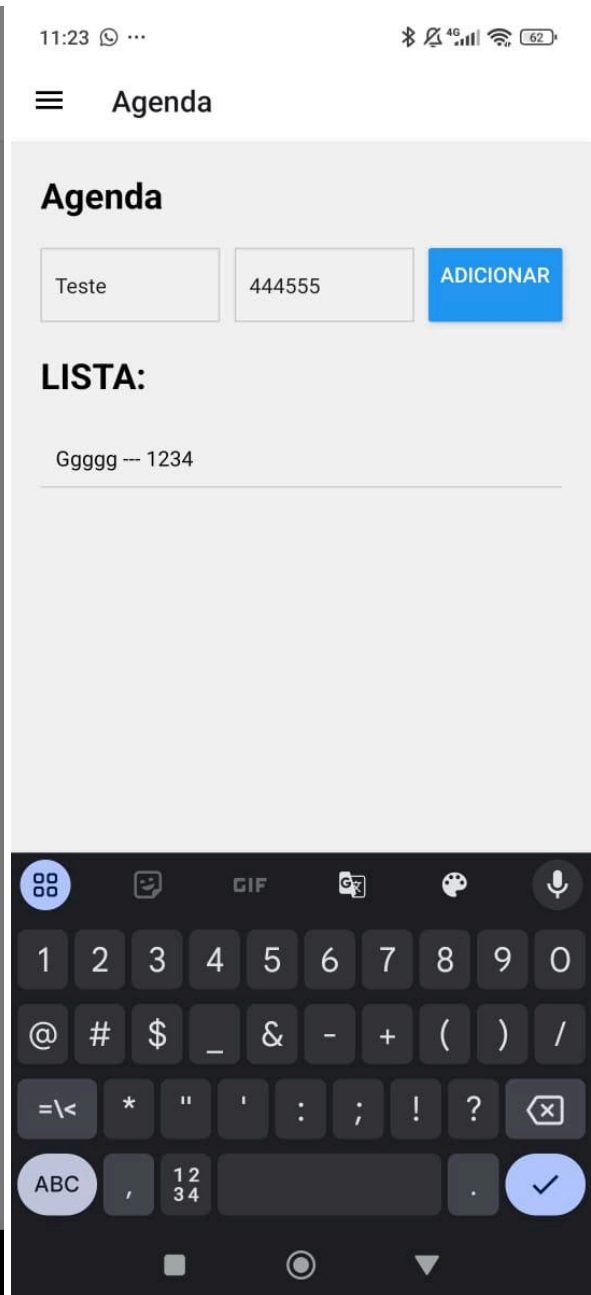
import 'react-native-gesture-handler'; //apenas para o nativo android

//const Tab = createBottomTabNavigator();
const Drawer = createDrawerNavigator();

export default function App() {
  return (
    <NavigationContainer>
      <Drawer.Navigator initialRouteName="Home">
        <Drawer.Screen name="Home" component={HomeScreen} />
        <Drawer.Screen name="Calculadora" component={CalculoScreen} />
        <Drawer.Screen name="Agenda" component={AgendaScreen} />
        <Drawer.Screen name="IMC" component={ImcScreen} />
      </Drawer.Navigator>
    </NavigationContainer>
  );
}
```

A principal mudança relevante é a adição do novo objeto, importado, 'agendaScreen' logo no inicio do código e o novo item no menu entre o 'calculoScreen' e o 'IMCScreen'.

Vejamos os resultados:



11:23



≡ Agenda

Agenda

ADICIONAR

LISTA:

Ggggg -- 1234

Teste -- 444555

A execução foi um sucesso.

Capítulo 9: Banco de dados, aplicando nosso primeiro exemplo.

NOME: _____ N° _____

TURMA: _____

PROFESSOR: _____

DATA: __/__/____

NOTA:

Exercícios

- 1 – Defina com suas palavras, o que é o 'repository'.
- 2 – O que aconteceria se, ao criar uma tabela nova no procedimento apresentado, a tabela já existisse?
- 3 – Descreva a importância de saber SQL neste contexto que estamos apresentando.
- 4 – Por que, ao fim do arquivo de repository, é chamado o createTable.
- 5 – No front, é utilizado um objeto para exibir os resultados das buscas, qual seria este objeto?

Atividade Prática

1 – Utilizando o exemplo que criamos, adicione um novo campo, o endereço em nossa agenda;

Capítulo 10: Banco de dados, Segunda parte.

10.1 Introdução.

Agora vamos acrescentar um novo fluxo, a remoção. Neste caso, vamos criar a função de deletar um registro de nossa tabela agenda.

Para isso, vamos ter que alterar alguns arquivos existentes, porém muito rapidamente podemos fazer tais alterações.

10.2 Alterando código.

10.2.1 Repository.

Bom, vamos começar com o 'agendaRepository.js', que contém as relações com nosso banco de dados, para tal, vamos adicionar a seguinte função:

```
const removerAgenda = (id, setAgenda) => {  
  db.transaction(tx => {  
    tx.executeSql(  
      'DELETE FROM agenda WHERE id = ?;',  
      [id],  
      () => {  
        console.log("Dados removidos com sucesso.");  
        getAgenda(setAgenda);  
      },  
      (tx, error) => {  
        console.log("Erro ao remover dados:", error);  
        return true;  
      }  
    );  
  });  
};
```

Criamos aqui um método chamado 'removerAgenda', este método recebe dois parâmetros, o 'id' que queremos deletar e o setAgenda para poder alterar a variável agenda do front.

Analisando este código, ele, como dito anteriormente, é bem parecido com os outros, onde inicialmente temos os mesmos comandos apresentados no capítulo anterior, tendo isso em vista, vamos aos detalhes.

O SQL está recebendo o parâmetro id como parte dele através dos parâmetros, visto na '?' que aparece no comando. No fluxo do caminho que deu certo, ou seja na parte do código que é executada quando a execução SQL funcionar, temos uma chamada "getAgenda(setAgenda);", esta chamada acontece chamando um outro método aqui deste arquivo, o 'getAgenda' que lista novamente, agora com o item deletado, e preenche a variável agenda através do 'setAgenda'.

Este fluxo é bem interessante e funciona maravilhosamente bem.

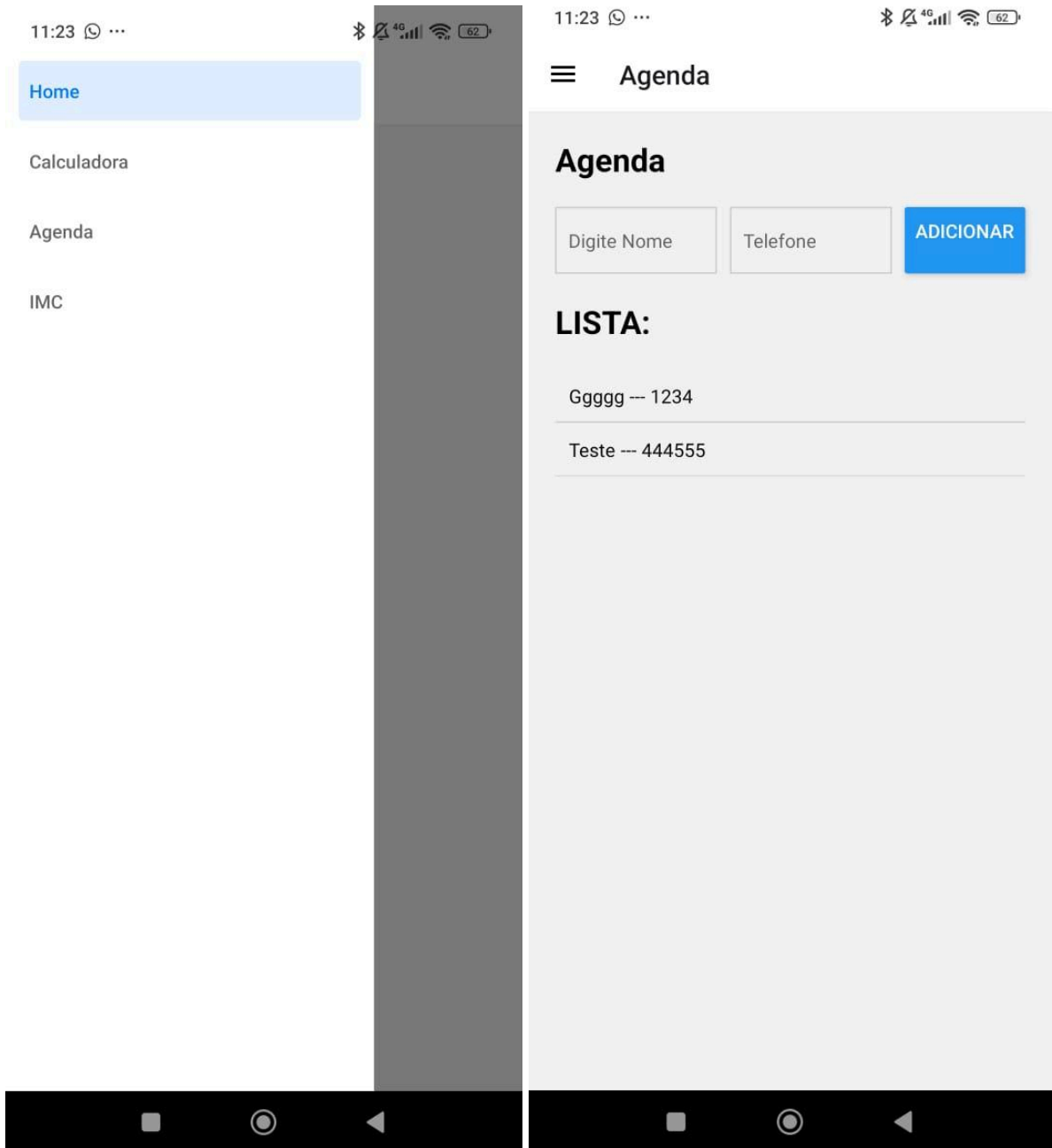
10.2.2 Front.

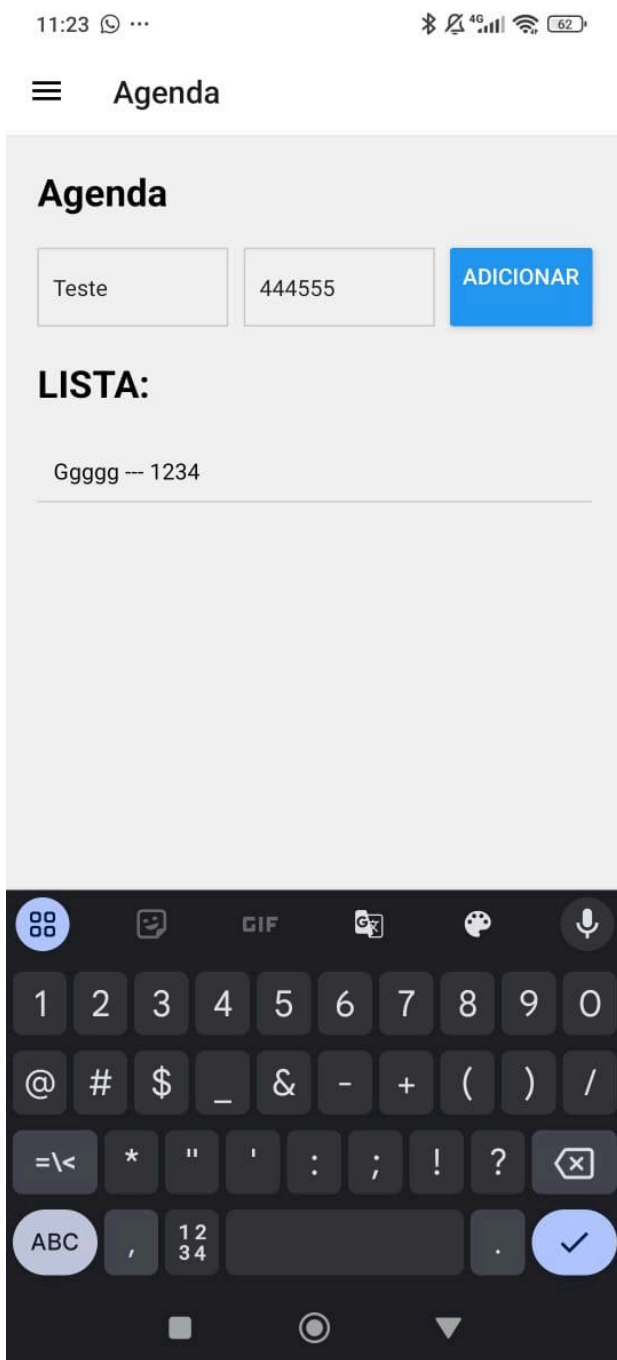
Agora vamos alterar o front, esta parte é muito fácil, visto que vamos apenas acrescentar um botão ao 'FlatList' que já existe. Vejamos o código:

```
</View>
  <Text style={styles.titulo}>LISTA:</Text>
  <FlatList
    data={agenda}
    keyExtractor={({item}) => item.id.toString()}
    renderItem={({ item }) => (
      <View style={styles.item}>
        <Text>{item.nome}</Text>
        <Text> --- </Text>
        <Text>{item.telefone}</Text>
        <Button title="Remover" onPress={() => removerAgenda(item.id,
setAgenda)} />
      </View>
    )}
  />
</View>
```

O código apresentado aqui em cima, basicamente o mesmo view que vimos no capítulo anterior, cuja função era listar o conteúdo, porém acrescentamos agora um 'Button', componente simples do react, onde temos o evento onPress associado ao método que criamos no arquivo repository, o 'removerAgenda' e nele passamos o 'id' do item que estamos visualizando e a variável setAgenda, para permitir a atualização do conteúdo das tabelas. Vemos que, o botão se encontra dentro da

estrutura de repetição interna do FlatList, sendo assim, o mesmo irá aparecer ao lado de cada uma das linhas da lista, passando um id diferente para cada botão permitindo a remoção individual de cada objeto. Vejamos a execução:





Executado com sucesso.

Capítulo 10: Banco de dados, Segunda parte.

NOME: _____ N° _____

TURMA: _____

PROFESSOR: _____

DATA: __/__/__

NOTA:

Exercícios

- 1 – Observando o repository ajustado, por que é chamado o getAgenda no fluxo do sucesso.
- 2 – Vendo o fluxo geral, como funciona a passagem do parâmetro id?
- 3 – Por que o parâmetro de entrada do método novo precisa que seja passado o setAgenda?
- 4 – Visto o front, como é feita a chamada do novo método?
- 5 – Como que, mesmo criando apenas um único botão, o mesmo se repete na tela com ids diferentes em sua chamada do método de remoção?

Atividade Prática

1 – Você deve melhorar a aplicação, primeiramente ajuste o CSS para que o botão e a distribuição da lista fique mais adequada a tela. Essa atualização de deletar deve estar também constando na tarefa anterior, aquela que consta o campo de endereço.

Tarefa Extra: Complete o crud.

NOME: _____ N° _____

TURMA: _____

PROFESSOR: _____

DATA: __/__/____

NOTA:

Atividade Prática

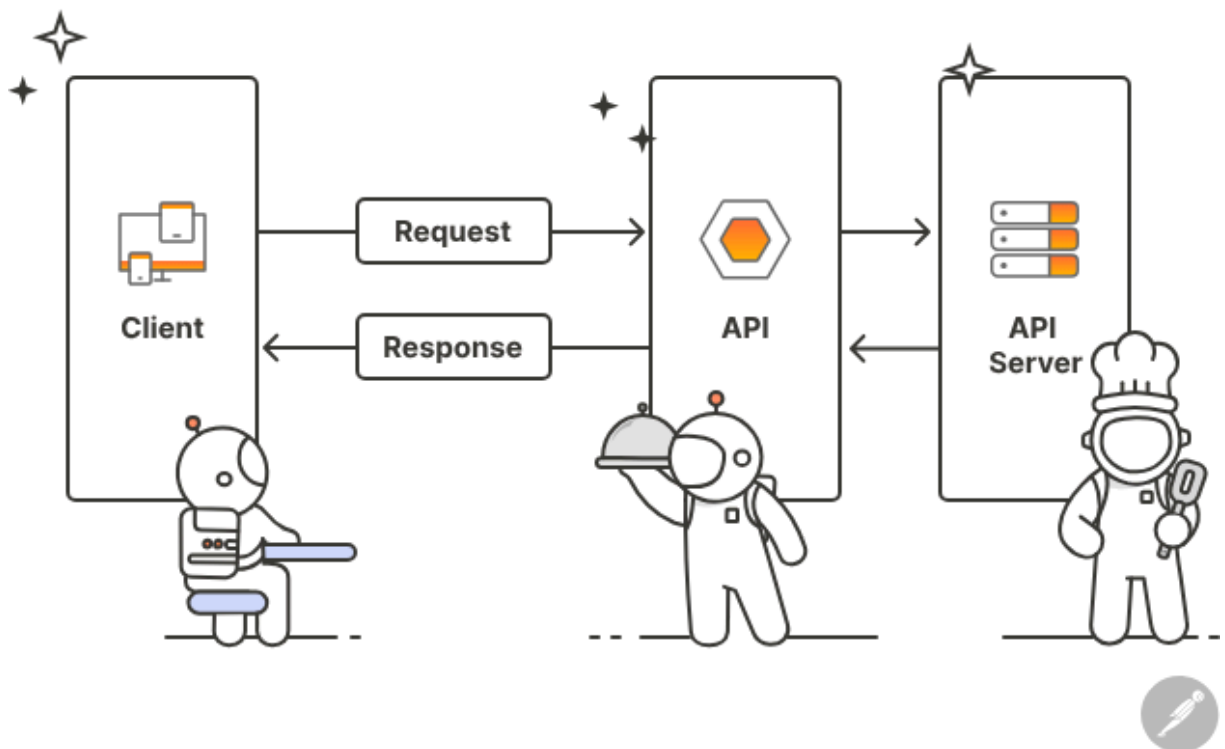
1 – Complete o crud, adicionando o que for necessário para uma função de editar os registros, ou seja faça com que seja possível alterar o registro já cadastrado.

Capítulo 11: Requisitando APIs.

11.1 Introdução.

Vamos começar a parte final, onde falaremos do de API's.

Uma API, ou Interface de Programação de Aplicações, é um conjunto de definições e protocolos que permite que diferentes sistemas de software interajam entre si. Imagine uma API como um garçom em um restaurante. Você, o cliente (ou aplicação), pede comida (dados ou funcionalidades) ao garçom (API). O garçom, por sua vez, comunica o seu pedido à cozinha (outro sistema de software) e traz de volta a sua comida (resposta).



As APIs simplificam e agilizam o desenvolvimento de software, permitindo que os programadores usem funcionalidades e dados de outros sistemas sem precisar entender os detalhes complexos de implementação. Isso economiza tempo e recursos, além de reduzir a duplicação de código.

Existem diferentes tipos de APIs, como APIs web, APIs de bibliotecas e APIs de sistema operacional. As APIs web são amplamente utilizadas para comunicação entre aplicações através da internet, utilizando protocolos como HTTP e formatos de dados como JSON e XML.

A popularidade das APIs tem crescido exponencialmente nos últimos anos, impulsionada pela necessidade de integração entre diferentes

plataformas e serviços online. Diversas empresas, como Google e Facebook, disponibilizam APIs públicas que permitem aos desenvolvedores criarem aplicações inovadoras e integradas aos seus serviços.

11.2 JSON

JSON, abreviação de JavaScript Object Notation, é um formato de dados leve e legível por humanos, amplamente utilizado para representar estruturas de dados e para troca de informações, especialmente em aplicações web. Pense em um arquivo JSON como uma receita de bolo: ele lista os ingredientes (dados) e suas quantidades (valores) de forma organizada e fácil de entender, tanto para você quanto para o computador.

A beleza do JSON está na sua simplicidade. Ele utiliza pares de chave-valor para representar dados, onde a "chave" é como um rótulo que descreve o dado e o "valor" é o próprio dado. Por exemplo: "nome": "Maria" indica que o valor associado à chave "nome" é "Maria". Essas informações são organizadas em objetos, que podem conter múltiplos pares chave-valor, e arrays, que são listas ordenadas de dados.

```
{  
  "titulo": "A Revolução dos Bichos",  
  "autor": "George Orwell",  
  "anoPublicacao": 1945,  
  "disponivel": true  
}
```

Este exemplo acima demonstra a simplicidade, onde passamos os dados via “chave”:”valor”. Este formato leve e expansível é perfeito para interagir com a transferência de dados entre aplicações diferentes entre aplicações diferentes.

11.3 Testando API

Vamos ao nosso exemplo simples, existe uma API disponibilizada pelos Correios brasileiros que, ao buscar um CEP qualquer, a mesma retorna os dados existentes deste CEP. Tal API está sob a seguinte URL: (<https://viacep.com.br/ws/11320000/json/>).

Podemos ver que, via URL é passado um CEP específico, no caso o ‘11320000’, podemos substituir o número por qualquer outro.

Ao executar essa URL obtemos a seguinte resposta:

```
{
  "cep": "11320-000",
  "logradouro": "Avenida Presidente Wilson",
  "complemento": "lado par",
  "unidade": "",
  "bairro": "Centro",
  "localidade": "São Vicente",
  "uf": "SP",
  "estado": "São Paulo",
  "regiao": "Sudeste",
  "ibge": "3551009",
  "gia": "6579",
  "ddd": "13",
  "siafi": "7121"
}
```

Observando, podemos ver a quantidade bem razoável de informação fornecida, informações estas que podem ser utilizadas em aplicações que nos for conveniente.

11.4 Exemplo de Código.

Vamos a aplicação deste código.

Começaremos criando um novo arquivo, para armazenar nossos códigos de chamadas de API, para nosso exemplo teremos apenas uma chamada, mas é interessante sempre deixar pronto para novas implementações. Vamos começar criando a pasta 'apis' e o arquivo 'apis.js'.



Na sequência vamos codificar este arquivo apis.js.


```

class Apis{
  static async buscaEndereco(cep, setEndereco) {
    try {
      const cepLimpo = cep.replace(/\D/g, "");
      if (cepLimpo.length !== 8) {
        alert('CEP inválido!');
        return;
      }
      const response = await fetch(`https://viacep.com.br/ws/${cepLimpo}/json/`);
      const data = await response.json();
      if (data.erro) {
        alert('CEP não encontrado!');
        return;
      }
      setEndereco(data);
    } catch (error) {
      console.error(error);
      alert('Erro ao buscar CEP!');
    }
  }
}
export default Apis;

```

Este código é bem puro, praticamente uma classe diretamente feita em JavaScript. Vamos observar o fluxo dela.

Uma classe chamada 'Apis' é criada e dentro dela um método chamado 'buscaEndereco' que recebe o 'CEP' e o setEndereco como parâmetro. Já fizemos isso em outros casos, ao passar o setEndereco, vamos carregar ele com o resultado de nossa busca para passar para o front mais adiante.

Na sequência, o fluxo de programação é bem limpo e começa removendo os possíveis espaços na variável CEP, criando uma variável chamada cepLimpo. Na sequência, faz uma validação, o CEP deve ter oito caracteres, logo isso pode ser validado antes de bater na API.

Por fim é chamada a API através do comando **fetch**.

Repara-se que, após a consulta o retorno é gravado na variável 'data' e validado, para saber se não houve nenhum erro. Se não houver, o mesmo é gravado no 'setEndereco' e o fluxo é finalizado.

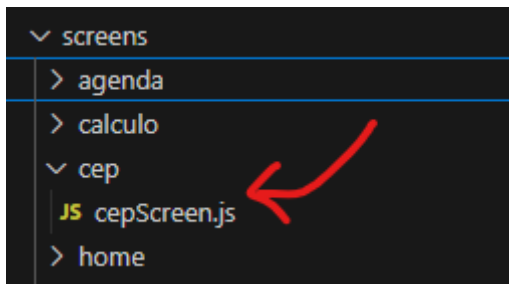
Reparamos em um comando diferente, o 'await' ao lado do comando da chamada da api, o 'fetch'. Este comando , junto com o 'async' no inicio

do método faz com que o procedimento não seja síncrono, ou seja, a chamada acontece sem travar o celular devido a ficarmos esperando a chamada ser respondida.

Com isso temos nosso método mais complexo.

Na sequência, vamos para a criação de uma nova tela, o 'cepScreen'.

Vejamos como fica a criação da pasta e do arquivo:



com o arquivo 'cepScreen' criado, vamos ao código dele:

```
import React, { useState } from 'react';
import { StatusBar } from 'expo-status-bar';
import { StyleSheet, View, Text } from 'react-native';
import TextInputBox from '../components/TextInputBox/TextInputBox';
import CustomButton from '../components/CustomButton/CustomButton';
import Apis from '../apis/apis';

function CepScreen() {
  const [cep, setCep] = useState("");
  const [endereco, setEndereco] = useState(null);

  return (
    <View style={styles.container}>
      <StatusBar style="auto" />

      <Text style={styles.title}>CEP</Text>
      <TextInputBox
        value={cep}
        onChangeText={setCep}
        placeholder="Digite o CEP"
        keyboardType="numeric"
      />
      <CustomButton
        title="Calcular"
        onPress={() => Apis.buscaEndereco(cep, setEndereco)}
        style={styles.button}
      />
    </View>
  );
}
```

```

    />

<Text style={styles.mensagem}>{endereco ? `Seu endereco:
${endereco.logradouro}` : 'Insira o CEP'}</Text>

    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    alignItems: 'center',
    justifyContent: 'center',
  },
  title: {
    fontSize: 24,
    marginBottom: 20,
  },
  picker: {
    height: 50,
    width: 200,
  },
  mensagem: {
    fontSize: 18,
    marginBottom: 20,
  }
});

export default CepScreen;

```

Está screen, como outras apresentadas ,não possui um comportamento muito diferente de qualquer outro modelo utilizado, porém vale ressaltar alguns pontos.

Podemos ver na linha que criamos a variável 'endereco' que quando usamos o 'useState' nela, não passamos o normal, vazio("") e sim o (null), isso indica que a variável é um objeto (JSON) não uma simples string.

Também vale a menção ao comando que exibe o resultado na tela:

<Text style={styles.mensagem}>{endereco ? `Seu endereco:

`{endereco.logradouro}` : **`Insira o CEP`**</Text>. O comando em si é bem simples, já fizemos um parecido, com validação de valor para exibir uma mensagem diferente se a variável 'endereco' não estiver preenchida, mas o interessante fica na exibição, no caso a variável endereco está sendo exibida da seguinte forma: `{endereco.logradouro}` por que estamos imprimindo na tela apenas o nome da rua. Caso queiramos outras informações, devemos observar o json de resposta que foi colocado lá em cima, todas aquelas informações podem ser utilizadas aqui apenas colocando o nome da chave em questão. Agora falta apenas a adição de mais um item no menu, alterando o APP.js como descrito abaixo:

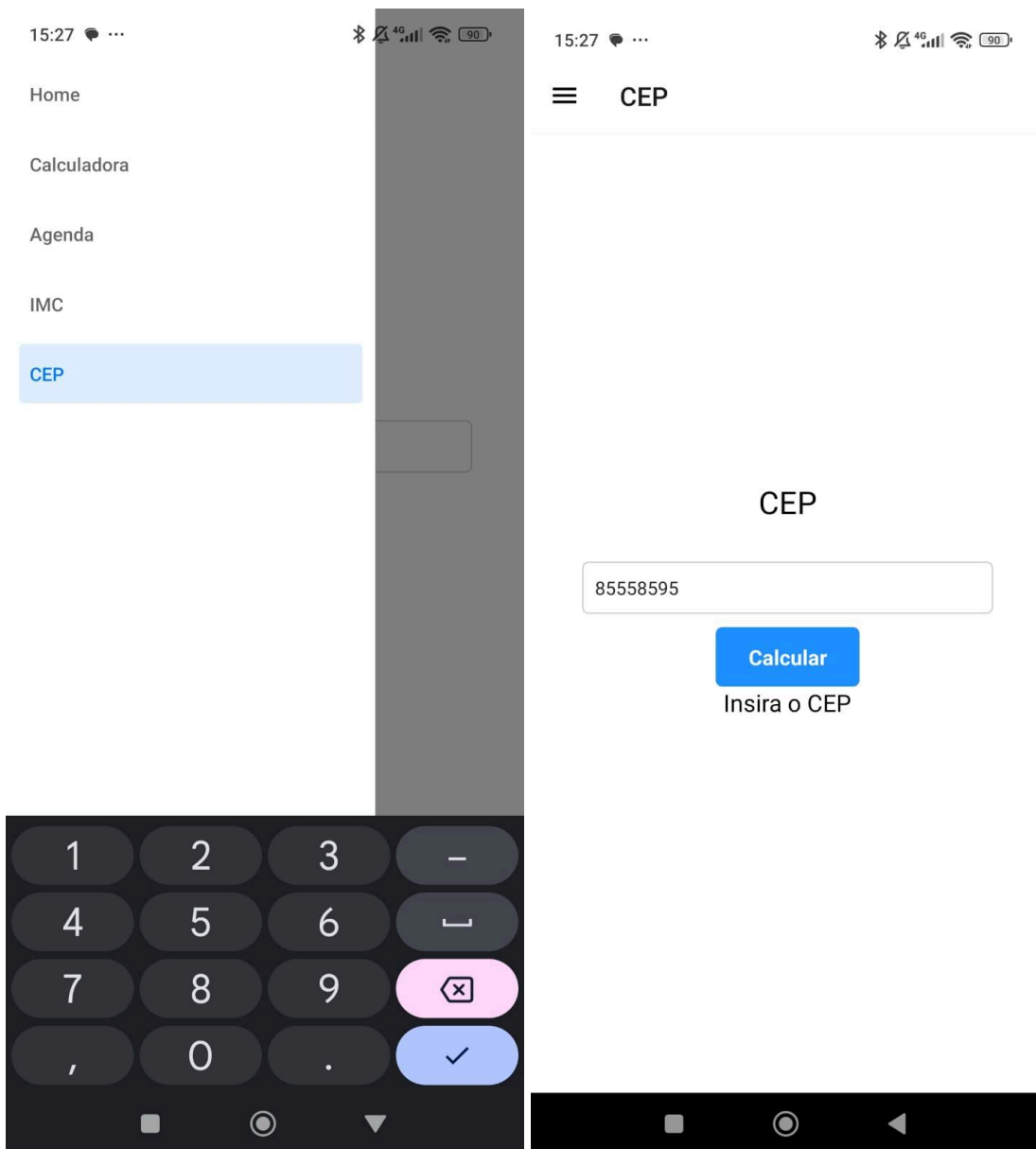
```
import { NavigationContainer } from '@react-navigation/native';
import { createDrawerNavigator } from '@react-navigation/drawer';
import CalculoScreen from './screens/calculo/CalculoScreen/';
import HomeScreen from './screens/home/HomeScreen/';
import ImcScreen from './screens/imc/ImcScreen/';
import AgendaScreen from './screens/agenda/agendaScreen/';
import CepScreen from './screens/cep/cepScreen/';

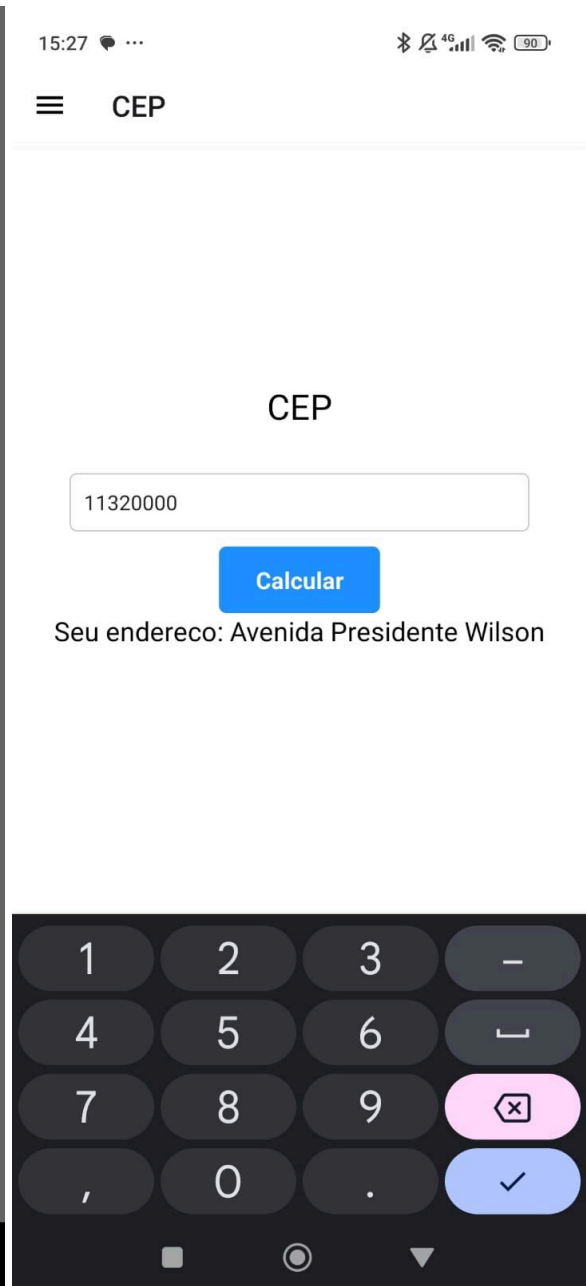
import 'react-native-gesture-handler'; //apenas para o nativo android

//const Tab = createBottomTabNavigator();
const Drawer = createDrawerNavigator();

export default function App() {
  return (
    <NavigationContainer>
      <Drawer.Navigator initialRouteName="Home"
      >
        <Drawer.Screen name="Home" component={HomeScreen} />
        <Drawer.Screen name="Calculadora" component={CalculoScreen}
        />
        <Drawer.Screen name="Agenda" component={AgendaScreen} />
        <Drawer.Screen name="IMC" component={ImcScreen} />
        <Drawer.Screen name="CEP" component={CepScreen} />
      </Drawer.Navigator>
    </NavigationContainer>
  );
}
```

No nosso caso, as linhas em negrito foram acrescentadas, adicionando a importação a screen do CEP e adicionando ela na lista do APP em execução. Vamos ao teste em APP.





Com isso temos nosso protótipo em execução.

Capítulo 11: Requisitando APIs.

NOME: _____ N° _____

TURMA: _____

PROFESSOR: _____

DATA: __/__/__

NOTA:

Exercícios

- 1 – Explique com suas palavras a função de uma API, seu objeto e momentos adequados para seu uso.
- 2 – Por que é necessário o uso do JSON como ferramenta em uma API.
- 3 – Busque na internet e explique as vantagens e desvantagens do uso do JSON em relação ao XML.
- 4 – Busque na internet e explique como um JSON pode passar vetores de dados em seu formato de texto.

Atividade Prática

1 – Para sua aplicação, vamos criar uma nova chamada a uma api, (<https://api.thecatapi.com/v1/images/search>) esta api retorna um JSON que contem uma URL de uma imagem de um gato aleatório, tente recuperar essa url e colocar esta imagem dentro de nosso aplicativo.